

Учреждения образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 1-40 05 01 Информационные системы и технологии
Специализация 1-40 05 01-03 Информационные системы и технологии (изда-
тельско-полиграфический комплекс)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА ДИПЛОМНОГО ПРОЕКТА НА ТЕМУ:

Веб-приложение для аренды жилья

Обучающийся	<u>4 курс, 2 группа</u>	_____	_____	<u>П.С. Синяк</u>
	курс, группа	дата	подпись	И. О. Ф.

Руководитель
дипломного проекта преп.-стажер. _____ А.В. Комкова
должность, ученая степень, ученое звание дата подпись И. О. Ф.

И.о. заведующего				
кафедрой	<u>К.Т.Н.</u>	<u> </u>	<u> </u>	<u>Е.А. Блинова</u>
	ученая степень, ученое звание	дата	подпись	И. О. Ф.

Консультант _____ ст. преп. _____ А.С. Соболевский
должность, ученая степень, ученое звание _____ дата _____ И. О. Ф.

Нормоконтролер _____ асс. _____ В. А. Алешаускас
 должность, ученая степень, ученое звание _____ дата _____ подпись _____ И. О. Ф.

Дипломный проект защищен с оценкой

Председатель ГЭК _____ к.т.н., доцент _____ В.К. Дюбков
 должность, ученая степень, ученое звание _____ дата _____ подпись _____ И. О. Ф.

Минск 2025

Министерство образования Республики Беларусь
Учреждение образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Информационных систем и технологий

УТВЕРЖДАЮ
И.о. заведующего кафедрой
Е.А. Блинова
« » 2025 г.

**ЗАДАНИЕ
НА ДИПЛОМНЫЙ ПРОЕКТ**

обучающемуся Синяку Павлу Сергеевичу
(фамилия, имя, отчество)

Курс 4 Группа 2

Специальность 1-40 05 01 Информационные системы и технологии

Специализация 1-40 05 01-03 Информационные системы и технологии (изда-
тельско-полиграфический комплекс)

Тема дипломного проекта: Веб-приложение для аренды жилья
утверждена ректором БГТУ 10 февраля 2025 г. № 32-С

Исходные данные к дипломному проекту:

– приложение должно поддерживать четыре роли: Гость, Администратор,
Пользователь, Владелец;

– функции пользователя с ролью «Гость»: регистрация и авторизация, филь-
трация, сортировка, поиск и просмотр информации о жилье;

– функции пользователя с ролью «Администратор»: создание и удаление кри-
териев и категорий жилья, блокировка, разблокировка и верификация пользо-
вателей и владельцев, просмотр статистика сервиса;

– функции пользователя с ролью «Пользователь»: бронирование жилья, под-
держка чата с владельцем, просмотр статистики бронирования жилья,
написание отзыва, изменение личной информации;

– функции пользователя с ролью «Владелец»: создание бронирования, добав-
ление, удаление и редактирование жилья, поддержка чата с пользователем,
одобрение и отмена бронирования, просмотр статистики.

Программная платформа и технологии: PostgreSQL 16, Node.js 20,
TypeScript 5, Express.js 4, React.js 18, Prisma ORM.

Краткое содержание расчетно-пояснительной записки:

- 1) оглавление;
- 2) реферат;
- 3) введение;
- 4) раздел 1: постановка задачи и аналитический обзор аналогичных решений;

- 5) раздел 2: проектирование веб-приложения
- 6) раздел 3: разработка веб-приложения;
- 7) раздел 4: тестирование веб-приложения;
- 8) раздел 5: руководство пользователя (руководство по эксплуатации);
- 9) раздел 6: технико-экономическое обоснование проекта;
- 10) заключение;
- 11) список использованных источников;
- 12) приложения и графическая часть;

Перечень графического материала

- 1) диаграмма вариантов использования;
- 2) архитектурная схема веб-приложения;
- 3) логическая схема базы данных;
- 4) блок-схема алгоритма добавления жилья;
- 5) блок-схема алгоритма бронирования жилья;
- 6) скриншот рабочего веб-приложения;
- 7) таблица экономических показателей.

Консультанты по проекту

Раздел	Консультант
Технико-экономическое обоснование проекта	Соболевский А.С.

Примерный календарный план выполнения дипломного проекта

№ п/п	Наименование структурного элемента дипломного проекта	Срок выполнения	Примечание
1	Постановка задачи и аналитический обзор аналогичных решений	24.03.2025 г.	
2	Проектирование веб-приложения	25.03.2025 г.	
3	Разработка веб-приложения;	29.03.2025 г.	
4	Тестирование веб-приложения;	30.04.2025 г.	
5	Руководство пользователя (руководство по эксплуатации);	07.05.2025 г.	
6	Технико-экономическое обоснование проекта;	24.05.2025 г.	
7	Рецензирование дипломного проекта	31.05.2025 г.	

Дата выдачи задания 24 марта 2025 г.

Срок сдачи законченного дипломного проекта 02 июня 2025 г.

Руководитель дипломного проекта А.В. Комкова
(подпись)

П.С. Сняк
(подпись обучающегося)

Дата « » 2025 г.

Оглавление

Реферат	6
Abstract	7
Введение.....	8
1 Постановка задачи и аналитический обзор аналогичных решений	9
1.1 Обзор аналогов	9
1.1.1 Приложение «Airbnb»	9
1.1.2 Приложение «Cian»	10
1.1.3 Приложение «Sutochno».....	11
1.2 Постановка задачи.....	12
1.3 Патентный поиск.....	13
1.4 Выводы по разделу.....	14
2 Проектирование веб-приложения.....	15
2.1 Описание средств разработки.....	15
2.1.1 Visual Studio Code	15
2.1.2 Платформа Node.js.....	16
2.1.3 Язык программирования TypeScript	16
2.1.4 Библиотека React	17
2.1.5 PostgreSQL	18
2.2 Разработка функциональных требований и вариантов использования.	18
2.3 Общая архитектура приложения	20
2.4 Проектирование базы данных.....	21
2.5 Проектирование основных алгоритмов	26
2.5.1 Алгоритм добавления жилья	27
2.5.2 Алгоритм бронирования жилья.....	28
2.6 Выводы по разделу.....	29
3 Разработка веб-приложения	30
3.1 Разработка серверной части.....	30
3.1.1 Регистрация	35
3.1.2 Безопасность.....	37
3.1.3 Авторизация	37
3.1.4 Управление жильем в системе	39
3.1.5 Бронирование жилья.....	43
3.1.6 Оставление отзывов с оцениванием после сделки.....	44
3.2 Разработка клиентской части.....	45
3.3 Выводы по разделу.....	48
4 Тестирование веб-приложения	49

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Оглавление		
Разраб.	Синяк П.С.						
Пров.	Комкова А.В.						
Н. контр.	Алешаускас В.А.						
Утв.	Блинова Е.А.				БГТУ 1-40 05 01, 2025		

4.1 Тестирование авторизации и регистрации	49
4.2 Тестирование добавление жилья	52
4.3 Тестирование поиска жилья.....	53
4.4 Тестирование изменения настроек пользователя	55
4.5 Тестирование бронирования жилья	56
4.6 Выводы по разделу.....	56
5 Руководство пользователя (руководство по эксплуатации)	57
5.1 Руководство для гостя	57
5.2 Руководство для пользователя.....	57
5.3 Руководство для владельца	61
5.4 Руководство для администратора.....	64
5.5 Установка приложения	67
5.6 Выводы по разделу.....	68
6 Техничко-экономическое обоснование проекта	69
6.1 Общая характеристика разрабатываемого программного средства	69
6.2 Исходные данные для проведения расчётов	70
6.3 Обоснование цены программного средства	71
6.3.1 Расчет затрат рабочего времени на разработку программного средства.....	71
6.3.2 Расчет основной заработной платы	72
6.3.3 Расчет дополнительной заработной платы	72
6.3.4 Расчет отчислений в Фонд социальной защиты населения и по обязательному страхованию	73
6.3.5 Расчет суммы прочих прямых затрат	73
6.3.6 Расчет суммы накладных расходов	74
6.3.7 Сумма расходов на разработку программного средства	74
6.3.8 Расходы на сопровождение и адаптацию.....	75
6.3.9 Расчет полной себестоимости	75
6.3.10 Определение цены, оценка эффективности	75
6.4 Выводы по разделу.....	77
Заключение	78
Список использованных источников	79
ПРИЛОЖЕНИЕ А Диаграмма вариантов использования	80
ПРИЛОЖЕНИЕ Б Архитектурная схема веб-приложения.....	81
ПРИЛОЖЕНИЕ В Логическая схема базы данных.....	82
ПРИЛОЖЕНИЕ Г Блок-схема алгоритма добавления жилья	83
ПРИЛОЖЕНИЕ Д Блок-схема алгоритма бронирования жилья	84
ПРИЛОЖЕНИЕ Е Скриншот рабочего веб-приложения	85
ПРИЛОЖЕНИЕ Ж Таблица экономических показателей	86

Реферат

Пояснительная записка дипломного проекта содержит 79 страниц пояснительной записки, 19 таблиц, 11 формул, 42 иллюстраций, 8 источников литературы и 7 приложений.

ВЕБ-ПРИЛОЖЕНИЕ, POSTGRESQL, NODE.JS, TYPESCRIPT, EXPRESS.JS, REACT.JS, PRISMA ORM

Целью дипломного проекта является разработка веб-приложения для аренды жилья, обеспечивающего удобное взаимодействие между пользователями, владельцами и администраторами платформы.

В первой главе проводится постановка задачи и аналитический обзор аналогичных решений по тематике дипломного проекта.

Вторая глава посвящена обзору средств разработки и содержит описание технологий, использованных во время выполнения проекта.

В третьей главе описывается процесс разработки, принципы функционирования и назначение созданных компонент проекта.

В четвертой главе описывается контрольный пример, с проведением тестирования, и показывается поведения системы при разных внештатных ситуациях.

В пятой главе описано руководство пользователя, позволяющее подробно понять интерфейс программного средства.

В шестой главе приводится расчет экономических показателей себестоимости программного средства.

В заключении подводятся итоги выполненной работы и оценивается достижение поставленной цели.

Объем графической части дипломного проекта составляет 1,25 листа А1.

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Реферат		
Разраб.	Синяк П.С.						
Пров.	Комкова А.В.						
Н. контр.	Алешаускас В.А.						
Утв.	Блинова Е.А.				БГТУ 1-40 05 01, 2025		
					Лит.	Лист	Листов
					У	1	1

Abstract

Explanatory note of the diploma project 79 pages of explanatory note, 19 tables, 11 formulas, 42 illustrations, 8 sources of literature and 7 appendices.

WEB APPLICATION, POSTGRESQL, NODE.JS, TYPESCRIPT, EXPRESS.JS, REACT.JS, PRISMA ORM

The diploma project aims to develop a web application for property rental, enabling smooth interaction among users, owners, and administrators

The first chapter outlines the problem statement and provides an analytical review of similar solutions related to the topic of the diploma project.

The second chapter is devoted to a review of development tools and describes the technologies used during the project.

The third chapter describes the development process, the principles of functioning and the purpose of the created project components.

The fourth chapter describes a test case, with testing, as well as a demonstration of the behavior of the system in various emergency situations.

The fifth chapter describes a user manual that allows you to understand in detail the interface of the software tool.

The sixth chapter provides the calculation of economic indicators and the cost of software developed in the framework of the graduation project.

The conclusion summarizes the results of the work performed and evaluates the achievement of the set goal.

The volume of the graphic part of the diploma project is 1.25 sheets A1.

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Abstract		
Разраб.	Синяк П.С.						
Пров.	Комкова А.В.						
Н. контр.	Алешаускас В.А.						
Утв.	Блинова Е.А.				БГТУ 1-40 05 01, 2025		
					Лит.	Лист	Листов
					У	1	1

Введение

В современном мире, где повышенная мобильность и требования к качеству жизни становятся нормой, аренда жилья превращается в сложный процесс для обеих сторон – арендаторов и арендодателей. Постоянное увеличение количества информации о доступной недвижимости, условиях съема и предпочтениях арендаторов подчеркивает необходимость эффективных инструментов для упрощения этого процесса. Веб-приложения, разработанные для таких целей, играют ключевую роль, предлагая удобные механизмы поиска и аренды жилья.

Идея разработки веб-приложения для аренды жилья возникла из анализа текущих проблем на рынке недвижимости. Отсутствие удобных и интуитивно понятных платформ, которые бы объединяли арендаторов и арендодателей, затрудняет поиск подходящих вариантов и коммуникацию между участниками сделки. Предполагается, что такое приложение сможет минимизировать временные и организационные затраты пользователей, предоставив им удобный инструмент для поиска, сравнения и аренды жилья.

На этапе планирования проекта определены основные цели и предполагаемый функционал приложения. Оно должно включать механизмы поиска недвижимости с возможностью фильтрации. Особое внимание будет уделено созданию простого и интуитивно понятного интерфейса, который обеспечит удобство использования для пользователей с разным уровнем технической подготовки. Это позволит сделать процесс поиска и аренды жилья максимально комфортным. Интерфейс будет адаптирован для работы на различных устройствах, включая мобильные.

Разработка приложения будет ориентирована на обеспечение надежности, производительности и масштабируемости системы. Планируется использовать современные подходы к проектированию архитектуры, включая принципы модульности и асинхронности, чтобы приложение могло адаптироваться к изменениям рыночных требований и новым запросам пользователей. Проект охватит полный цикл разработки: от анализа потребностей рынка и проектирования структуры данных до реализации функционала и тестирования системы.

Создаваемое веб-приложение призвано стать универсальным решением, которое облегчит процесс аренды жилья, повысит прозрачность взаимодействия между сторонами и улучшит пользовательский опыт. В перспективе оно может стать важным инструментом на рынке недвижимости, помогая арендаторам находить жилье, соответствующее их ожиданиям, а арендодателям – эффективно управлять своими объектами жилья и выстраивать долгосрочные отношения с жильцами.

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	2 Работа редактора над журнальным изданием		
Разраб.		Иванов					
Пров.		Руково-					
Н. контр.		Нистю					
Утв.		Бли-			БГТУ 1-40 05 01, 2025		
					Лит.	Лист	Листов
					У	1	1

1 Постановка задачи и аналитический обзор аналогичных решений

Для разработки программного средства первоначально требуется проанализировать существующие аналоги, выделив их ключевые преимущества и недостатки, а также определив основные характеристики приложений данного типа. На основе этого анализа формируются задачи для создания программы. Все указанные шаги были реализованы в данной главе.

1.1 Обзор аналогов

Обзор аналогов – важный этап перед началом разработки приложения. Этот процесс позволяет выявить функции, которые приложение должно обязательно включать, определить уникальные возможности, которые станут его отличительной чертой.

На рынке уже существует множество платформ для аренды жилья. Однако, многие из них имеют недостатки: ограниченная кастомизация поиска, сложность управления объявлениями для владельцев или недостаточное взаимодействие между арендаторами и арендодателями. Это открывает возможность для создания продукта, который предложит гибкий функционал, простой интерфейс и эффективное взаимодействие между пользователями.

1.1.1 Приложение «Airbnb»

Airbnb [1] – это одна из ведущих платформ для поиска и бронирования жилья, которая предоставляет пользователям возможность находить подходящие варианты аренды, используя удобные фильтры, просматривать подробные описания объектов и бронировать жилье в режиме онлайн. Пример интерфейса данного сервиса представлен на рисунке 1.1 для ознакомления.

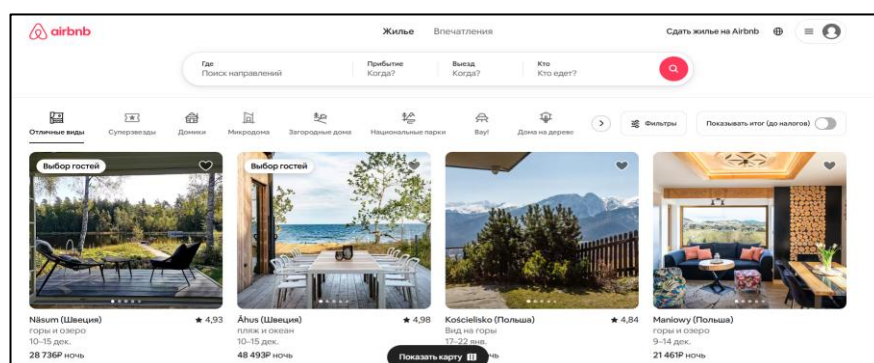


Рисунок 1.1 – Интернет-ресурс «Airbnb»

					ДП 01.00.ПЗ		
		ФИО	Подпись	Дата			
Разраб.	Синяк П.С.				1 Постановка задачи и аналитический обзор аналогичных решений		
Пров.	Комкова А.В.						
Н. контр.	Алешаускас В.А.						
Утв.	Блинова Е.А.						
					Лит.	Лист	Листов
					У	1	6
					БГТУ 1-40 05 01, 2025		

Одной из ключевых функций Airbnb является поиск жилья. Пользователи могут искать объекты недвижимости, применяя фильтры по местоположению, дате, цене, количеству гостей, рейтингу, удобствам и другим параметрам. Сервис также предоставляет персонализированные рекомендации на основе истории поиска и ранее забронированных объектов.

Для удобства пользователей Airbnb предлагает возможность создавать учетные записи, где можно управлять бронированиями, сохранять интересные объекты в «Избранное» и оставлять отзывы после завершения проживания. Пользователи могут ознакомиться с отзывами других гостей, что помогает сделать более информированный выбор при аренде жилья.

Для владельцев недвижимости Airbnb предоставляет платформу для регистрации и управления своими объектами. Они могут добавлять описания, фотографии, устанавливать цены, задавать правила отмены бронирования, а также управлять календарем доступности. Сервис предоставляет статистику по каждому объекту, включая количество просмотров, бронирований и средний рейтинг, что помогает владельцам анализировать спрос и оптимизировать свои предложения.

С технической стороны Airbnb использует сложные алгоритмы поиска и фильтрации, а также высокопроизводительные базы данных для обработки миллионов запросов пользователей ежедневно.

Эта платформа выделяется своей глобальной доступностью, поддержкой множества языков и валют, а также адаптивным интерфейсом, который обеспечивает удобство использования как на компьютерах, так и на мобильных устройствах. Благодаря широкому функционалу и пользовательскому комфорту, Airbnb остается одним из лидеров рынка онлайн-бронирования жилья.

1.1.2 Приложение «Cian»

Cian [2] – это одна из крупнейших платформ для поиска и аренды недвижимости, которая предоставляет пользователям возможность находить подходящие варианты жилья, используя гибкие фильтры, просматривать подробные описания объектов и напрямую связываться с арендодателями. Пример интерфейса данного сервиса представлен на рисунке 1.2 для ознакомления.

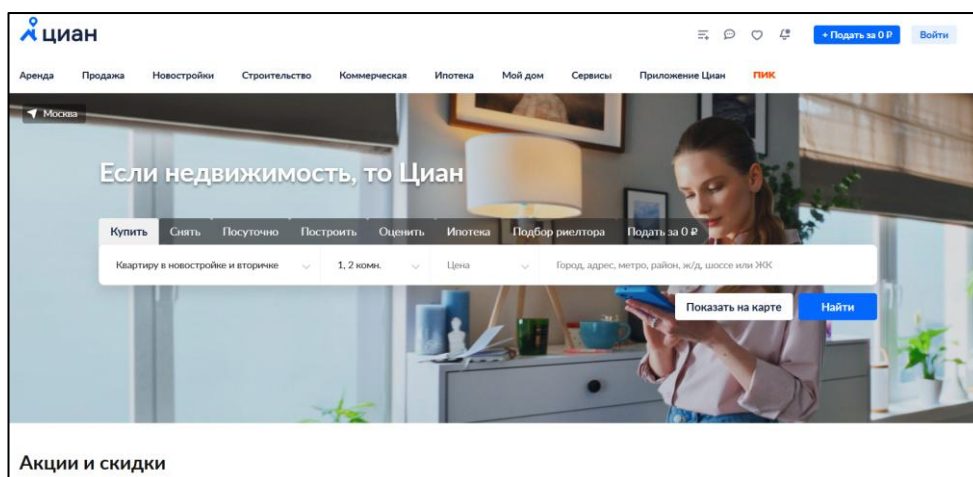


Рисунок 1.2 – Интернет-ресурс «Cian»

Ключевой функцией Cіan является поиск недвижимости. Пользователи могут находить объекты, применяя разнообразные фильтры: по цене, району, типу жилья, площади, количеству комнат, сроку аренды и другим параметрам. Также платформа предоставляет удобную карту, где объекты отображаются с указанием точного местоположения, что упрощает выбор подходящего варианта.

Для удобства пользователей Cіan предлагает возможность создавать личные кабинеты, где можно сохранять интересующие объявления в «Избранное», управлять своими объявлениями (для арендодателей) и отслеживать статус откликов.

Для арендодателей и продавцов недвижимости Cіan предоставляет функционал для размещения и управления объявлениями. Владельцы могут добавлять фотографии, описания, указывать стоимость и условия аренды, а также редактировать или удалять объекты. Платформа предоставляет статистику для каждого объявления, включая количество просмотров и откликов, что помогает анализировать спрос и корректировать предложения для повышения их привлекательности.

Удобный и интуитивно понятный интерфейс Cіan адаптирован для работы как на компьютерах, так и на мобильных устройствах, благодаря чему пользователи могут быстро находить и просматривать предложения в любое время. Благодаря обширной базе данных объектов, гибкости поиска и дополнительным функциям, Cіan остается одним из лидеров среди платформ для аренды и покупки недвижимости.

Значительная часть объявлений на Cіan размещается риелторскими агентствами, что иногда затрудняет поиск предложений от частных лиц. Пользователи, предпочитающие работать напрямую с владельцами, могут столкнуться с необходимостью фильтровать множество агентских объявлений.

1.1.3 Приложение «Sutochno»

Sutochno [3] – это онлайн-платформа, специализирующаяся на краткосрочной аренде жилья. Она предоставляет пользователям удобные инструменты для поиска и бронирования квартир, домов, комнат и других объектов недвижимости по всей стране. Пример интерфейса данного сервиса представлен на рисунке 1.3.

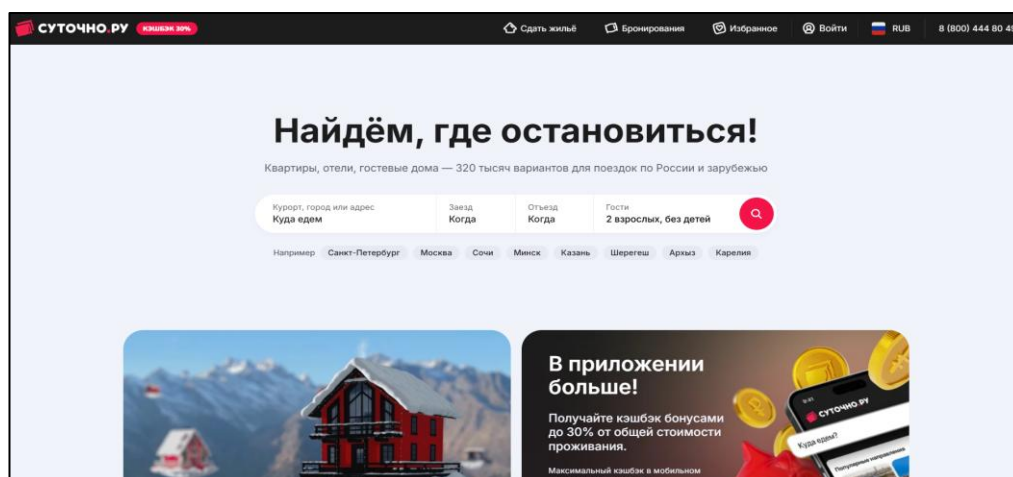


Рисунок 1.3 – Интернет-ресурс «Sutochno»

Основной функционал Sutochno сосредоточен на быстром поиске и аренде жилья на сутки или более короткие сроки. Пользователи могут искать объекты, применяя фильтры по цене, дате, количеству комнат, типу жилья, местоположению и другим параметрам. Система также предлагает карту для визуального поиска, на которой объекты отмечены с указанием стоимости аренды.

Платформа позволяет пользователям регистрировать личные кабинеты, где можно управлять бронированиями, сохранять интересные объекты в «Избранное» и оставлять отзывы после завершения проживания. Пользователи могут общаться с арендодателями через встроенный чат.

Для арендодателей Sutochno предоставляет возможность размещать и редактировать объявления. Владельцы могут добавлять описания, фотографии, указывать правила проживания, условия отмены бронирования и доступные даты. Платформа поддерживает аналитические инструменты, позволяющие владельцам отслеживать популярность их объектов: количество просмотров и бронирований.

Sutochno выделяется своей ориентацией на рынок, предлагая широкий выбор объектов в крупных городах и популярных туристических направлениях. Благодаря функционалу, направленному на удобство и прозрачность для всех участников, платформа остается востребованным инструментом как для арендаторов, так и для арендодателей, обеспечивая комфортное взаимодействие и безопасные сделки.

1.2 Постановка задачи

В процессе анализа существующих приложений-аналогов, были выявлены ключевые аспекты, которые обеспечивают удобство, функциональность и эффективность взаимодействия между арендаторами и арендодателями на рынке недвижимости. Исходя из сделанных выводов, в проектируемом веб-приложении необходимо реализовать современные и востребованные возможности, которые удовлетворят потребности пользователей и повысят их удовлетворенность процессом аренды жилья. Цель проекта – разработка современного веб-приложения, предоставляющего удобные инструменты для поиска, бронирования и управления недвижимостью, с акцентом на интуитивно понятный интерфейс, надежность и гибкость системы.

Для достижения поставленной цели приложение должно соответствовать следующим требованиям:

Должны быть выполнены следующие требования:

- обеспечивать возможность регистрации и авторизации;
- поддерживать роли: администратор, владелец, пользователь;
- поиск, фильтрация и сортировку жилья;
- возможность создавать и удалять критерии и категории жилья;
- возможность создавать пользователей, менять их статус, верифицировать для того, чтобы они могли создавать жилье;
- возможность создавать, удалять и редактировать жилье в системе;
- возможность создавать бронирования, менять их статус;

- поддержка чата между пользователем и владельцем;
- возможность просматривать информацию о жилье;
- возможность бронировать интересующее его жилье;
- возможность оставлять отзывы с оцениванием после сделки.

1.3 Патентный поиск

В данном подразделе представлены результаты патентного исследования, связанного с веб-приложениями для аренды жилья. По итогам проведенного поиска были выявлены патенты, описывающие технологии с функционалом, схожим с разрабатываемым веб-приложением для аренды жилья. Они представлены ниже в таблицах 1.1 и 1.2.

Таблица 1.1 – Описание патента №1

Наименование	Номер патента	Опубликовано	Авторы
«System and method for real estate data management and rental platform»	US20200380594A1	03.12.2020	John Smith, Emily Johnson

Данный патент описывает систему управления данными недвижимости и платформу для аренды жилья, реализованную в виде веб-приложения. Технология позволяет пользователям искать доступные объекты для аренды, фильтровать их по параметрам (местоположение, цена, количество комнат и др.) и взаимодействовать с арендодателями через встроенную систему обмена сообщениями. Платформа поддерживает автоматизированное создание договоров аренды на основе шаблонов, а также интеграцию с платежными системами для безопасного проведения транзакций. Система включает функцию рекомендаций, которая использует алгоритмы машинного обучения для подбора объектов, соответствующих предпочтениям пользователя. Дополнительно поддерживается визуализация объектов недвижимости с помощью виртуальных туров, что повышает удобство выбора жилья.

Таблица 1.2 – Описание патента №2

Наименование	Номер патента	Опубликовано	Авторы
« Online platform for property rental with augmented reality features»	US20180159838A1	07.06.2018	Michael Brown, Anna Lee

Патент описывает веб-приложение для аренды жилья с использованием технологий дополненной реальности (AR). Пользователи могут просматривать объекты недвижимости через веб-интерфейс, который интегрирован с AR-устройствами или мобильными приложениями. Технология позволяет визуализировать интерьеры объектов в реальном времени, добавляя

виртуальные элементы, такие как мебель или декор, для оценки соответствия жилья потребностям арендатора. Платформа также предоставляет функции для автоматического расчета арендной платы на основе рыночных данных, управления бронированиями и уведомления пользователей о новых предложениях. Система поддерживает интеграцию с социальными сетями для упрощения обмена информацией об объектах и отзывах арендаторов.

1.4 Выводы по разделу

В данном разделе был проведен обзор аналогичных веб-приложений, таких как Airbnb, Cian и Sutochno, что позволило выявить ключевые функции, которые должны быть реализованы в нашем приложении. Эти платформы предоставляют пользователям широкий спектр возможностей: от регистрации и авторизации до поиска и бронирования жилья с использованием удобных фильтров, а также управления объектами недвижимости для владельцев. Также был проведен патентный поиск.

Особое внимание уделено функционалу для различных ролей пользователей. Для владельцев объектов аренды важными элементами являются инструменты для добавления, редактирования и удаления объектов, управления бронированиями, а также ведения переписки с арендаторами. Пользователям необходимы функции просмотра и поиска жилья, бронирования, отслеживания статуса бронирований, а также возможности оставлять отзывы. Также определены требования к удобному интерфейсу, который должен быть интуитивно понятным и обеспечивать эффективное взаимодействие между владельцами недвижимости, арендаторами и администраторами системы для достижения высокой удовлетворенности пользователей.

В результате проведенного анализа аналоговых платформ было решено реализовать следующие пользовательские роли в проектируемом приложении:

- администратор, который отвечает за администрирование учетных записей, модерирование контента и управление системой в целом;
- владелец, который создает и поддерживает информацию о жилье, может создавать бронирования на определенные даты, одобрять или отклонять запросы на бронирование от пользователей, а также поддерживать чат с арендаторами;
- пользователь, который создает бронирования на выбранные даты, оставляет отзывы и оценки о жилье после проживания, а также ведет переписку с владельцем через встроенный чат;
- гость, который имеет доступ к каталогу жилья, может выполнять поиск по названию, сортировать и фильтровать объекты в каталоге, а также просматривать подробную информацию о жилье без необходимости регистрации.

Реализация данных ролей обеспечит четкое распределение функционала между участниками системы, что повысит удобство использования и эффективность взаимодействия в процессе аренды жилья.

2 Проектирование веб-приложения

2.1 Описание средств разработки

При разработке приложения необходимо выбрать языки программирования, инструменты и технологии, подходящие для реализации клиентской и серверной частей. Правильный выбор средств разработки может значительно упростить и ускорить создание проекта, а также минимизировать риски возникновения проблем в работе готовой системы. Неправильный выбор технологий, напротив, может привести к сложностям на этапе разработки и неполадкам в функционировании приложения. В данном разделе будут рассмотрены различные библиотеки, технологии и подходы, которые планируется использовать при создании приложения.

2.1.1 Visual Studio Code

Visual Studio Code – это популярная интегрированная среда разработки, которая завоевала доверие миллионов разработчиков по всему миру. Она поддерживает широкий спектр языков программирования, включая JavaScript, TypeScript, Python, C++ и многие другие, а также такие технологии, как React и Node.js. Благодаря своей универсальности, VS Code подходит как для начинающих, так и для опытных разработчиков, работающих над проектами любой сложности.

Одной из ключевых особенностей VS Code является его легковесность и высокая производительность. Среда быстро запускается и работает даже на не самых мощных компьютерах, что делает ее идеальным выбором для разработчиков, ценящих скорость и эффективность. При этом VS Code обладает гибкостью, позволяя пользователям настраивать интерфейс, горячие клавиши и функциональность под свои потребности и предпочтения.

VS Code предоставляет мощный набор встроенных инструментов для написания, отладки и тестирования кода. Встроенный терминал, поддержка систем контроля версий (например, Git) и интеллектуальная подсказка кода значительно упрощают процесс разработки. Кроме того, среда поддерживает отладку для множества языков, что позволяет быстро находить и исправлять ошибки в коде.

Особое внимание стоит уделить экосистеме расширений VS Code. В магазине расширений доступны тысячи плагинов, которые добавляют поддержку новых языков, фреймворков, баз данных и инструментов. Например, расширения для работы с Docker, Kubernetes или базами данных, такими как PostgreSQL и MongoDB, делают VS Code универсальным инструментом для всех этапов разработки – от написания кода до его тестирования и деплоя.

					ДП 02.00.ПЗ		
		ФИО	Подпись	Дата			
Разраб.	Синяк П.С.				2 Проектирование веб-приложения	Лит.	Лист
Пров.	Комкова А.В.					У	1
						Листов	
Н. контр.	Алешаускас В.А.					15	
Утв.	Блинова Е.А.					БГТУ 1-40 05 01, 2025	

В итоге, Visual Studio Code – это универсальная, гибкая и легковесная среда разработки, которая помогает разработчикам эффективно справляться с задачами любой сложности. Благодаря поддержке множества языков, встроенным инструментам и богатой экосистеме расширений, VS Code остается одним из лучших выборов для современных разработчиков [4].

2.1.2 Платформа Node.js

Node.js – это серверная среда выполнения, которая позволяет запускать JavaScript-код за пределами браузера. Основанная на движке V8, используемом в Google Chrome, Node.js обеспечивает высокую производительность и открывает возможности для создания полноценных серверных приложений. Эта технология изменила подход к разработке, позволив использовать JavaScript как для клиентской, так и для серверной части приложений.

Одной из главных особенностей Node.js является ее асинхронная, неблокирующая модель ввода-вывода, основанная на событиях. Такая архитектура делает Node.js идеальным выбором для создания масштабируемых веб-приложений, способных обрабатывать большое количество одновременных запросов. Это особенно важно для приложений, требующих высокой производительности, таких как чат-системы, онлайн-игры или платформы для обмена сообщениями в реальном времени.

Node.js также славится своей гибкостью и обширной экосистемой. Благодаря npm (Node Package Manager), разработчики имеют доступ к тысячам библиотек и модулей, которые значительно ускоряют процесс разработки. От фреймворков, таких как Express, до инструментов для работы с базами данных и тестирования – npm предоставляет все необходимое для создания современных приложений [5].

Подводя итог, Node.js – это универсальная и эффективная платформа, идеально подходящая для создания высокопроизводительных и масштабируемых веб-приложений. Благодаря поддержке асинхронной модели, богатой экосистеме npm и возможности использовать JavaScript на сервере, Node.js остается ключевым инструментом в арсенале современных разработчиков.

2.1.3 Язык программирования TypeScript

TypeScript – это язык программирования, представляющий собой надстройку над JavaScript, которая вводит статическую типизацию. Разработанный компанией Microsoft, TypeScript призван повысить предсказуемость и безопасность разработки, позволяя выявлять потенциальные ошибки на этапе компиляции. Это делает его мощным инструментом для создания надежных и масштабируемых приложений.

Основное преимущество TypeScript – это строгая проверка типов, которая помогает разработчикам обнаруживать ошибки еще до выполнения кода. Благодаря статической типизации код становится более читаемым и структурированным, что особенно важно для крупных проектов. TypeScript облегчает поддержку

и масштабирование приложений, минимизируя вероятность ошибок, связанных с динамической природой JavaScript, и повышая надежность системы.

TypeScript полностью совместим с JavaScript, что позволяет использовать все его стандартные возможности. При этом TypeScript добавляет дополнительные функции, такие как интерфейсы, перечисления и пользовательские типы данных. Эти инструменты способствуют созданию более структурированного и понятного кода, упрощая совместную работу в командах и долгосрочное сопровождение проектов [6].

В завершение, TypeScript – это продвинутый язык, который расширяет возможности JavaScript, делая разработку более безопасной, удобной и эффективной. Благодаря поддержке статической типизации и дополнительных функций, TypeScript стал стандартом для создания сложных и масштабируемых приложений, особенно в крупных командах, обеспечивая надежность кода.

2.1.4 Библиотека React

React – это популярная JavaScript-библиотека, разработанная компанией Facebook, предназначенная для создания современных и динамичных пользовательских интерфейсов. Благодаря своей гибкости и компонентному подходу, React стал одним из ключевых инструментов в арсенале веб-разработчиков, особенно для построения одностраничных приложений (SPA).

Основой React является компонентный подход, при котором интерфейс разбивается на независимые, переиспользуемые компоненты. Каждый компонент управляет своим состоянием и поведением, что упрощает разработку, тестирование и поддержку кода. Такой подход особенно ценен при создании сложных интерфейсов, где требуется высокая степень модульности и масштабируемости.

Одной из главных особенностей React является использование виртуального DOM (Document Object Model). Эта технология позволяет библиотеке минимизировать прямые манипуляции с реальным DOM, обновляя только те элементы интерфейса, которые действительно изменились. В результате React обеспечивает высокую производительность и отзывчивость приложений, улучшая пользовательский опыт даже в условиях интенсивных обновлений интерфейса.

React также отличается простотой интеграции с другими инструментами и библиотеками, что делает его универсальным решением для веб-разработки. Благодаря активной поддержке сообщества и богатой экосистеме, React предоставляет разработчикам все необходимое для создания современных, быстрых и интерактивных веб-приложений с высокой производительностью [7].

По итогам анализа, React – это гибкая и эффективная библиотека, упрощающая создание динамичных пользовательских интерфейсов. Компонентный подход, виртуальный DOM и широкая поддержка сообщества делают React идеальным выбором для разработки современных веб-приложений, от простых сайтов до сложных одностраничных приложений, обеспечивая удобство и надежность.

2.1.5 PostgreSQL

PostgreSQL – это объектно-реляционная система управления базами данных (СУБД) с открытым исходным кодом, которая заслужила репутацию одного из самых надежных и универсальных решений для хранения и обработки данных. Благодаря своей гибкости и мощным возможностям, PostgreSQL широко используется в высоконагруженных веб-приложениях и корпоративных системах.

Одной из ключевых особенностей PostgreSQL является поддержка ACID-транзакций, обеспечивающих надежность и консистентность данных даже в условиях сложных операций. СУБД поддерживает сложные запросы, разнообразные типы данных, включая географические, а также предоставляет возможности для создания хранимых процедур и триггеров. Это делает PostgreSQL идеальным выбором для приложений, требующих высокой степени функциональности и точности.

PostgreSQL отличается высокой производительностью и масштабируемостью, что позволяет эффективно обрабатывать большие объемы данных. Благодаря своей архитектуре, СУБД легко справляется с нагрузками, характерными для крупных веб-приложений и аналитических систем. Кроме того, PostgreSQL поддерживает расширяемость, позволяя разработчикам добавлять собственные функции и модули для решения специфических задач приложения.

Гибкость PostgreSQL делает его подходящим как для небольших стартапов, так и для крупных корпоративных решений. Активное сообщество и обширная документация обеспечивают поддержку и постоянное развитие системы, что делает ее надежным выбором для современных приложений [8].

В результате, PostgreSQL – это надежная, масштабируемая и универсальная СУБД, сочетающая высокую производительность и гибкость. Поддержка сложных запросов, ACID-транзакций и расширяемость делают PostgreSQL незаменимым инструментом для разработки современных приложений, работающих с большими объемами данных и требующих стабильности.

2.2 Разработка функциональных требований и вариантов использования

Приложение должно предоставлять функционал для регистрации и авторизации, позволяя пользователям идентифицировать себя и получать доступ к возможностям в зависимости от их роли: администратор, владелец недвижимости или обычный пользователь системы.

Администратор получит возможность управлять системой, включая редактирование категорий и критериев для поиска жилья, управление пользователями и их доступом, а также просмотр и анализ статистики по всем объектам и бронированиям.

Владелец недвижимости сможет добавлять, редактировать и удалять свои объекты аренды, управлять статусами бронирований, а также просматривать подробную статистику по своим объектам, включая количество просмотров, бронирований и отзывы. Для удобства взаимодействия с пользователями

владельцу будет доступен встроенный чат, через который можно уточнять детали бронирования и оперативно отвечать на вопросы арендаторов.

Пользователь сможет искать жилье, используя удобные фильтры и сортировку, бронировать интересные варианты, просматривать статус своих бронирований и историю операций. Для улучшения качества сервиса пользователь сможет оставлять отзывы и оценки после завершения аренды, а также управлять своими личными данными через настройки профиля.

На рисунке 2.1 и в приложении А представлена диаграмма вариантов использования, отражающая функциональные возможности системы.

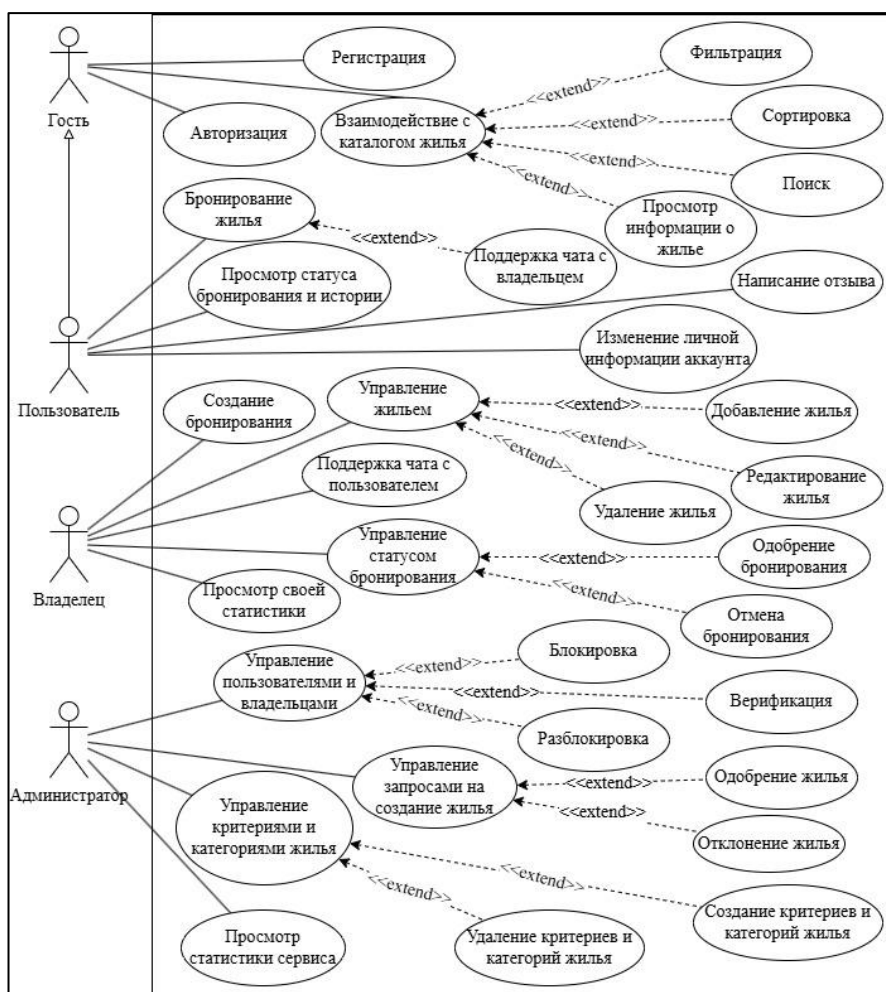


Рисунок 2.1 – Диаграмма вариантов использования

Рассмотрим основные прецеденты и их доступность для ролей.

Регистрация и авторизация:

- создание учетной записи доступно для гостя;
- вход в систему доступен для гостя.

Управление системой:

- редактирование категорий жилья доступно для администратора;
- редактирование критериев жилья доступно для администратора;
- управление учетными записями доступно для администратора;
- управление доступом пользователей доступно для администратора;
- просмотр статистики сервиса доступен для администратора.

Управление жильем:

- добавление жилья доступно для владельца;
- редактирование жилья доступно для владельца;
- удаление жилья доступно для владельца;
- управление жильем доступно для владельца;
- просмотр статистики по объектам жилья доступно для владельца.

Работа с жильем и бронированиями:

- взаимодействие с каталогом жилья через поиск с использованием фильтров и сортировки доступно для пользователя и гостя;
- просмотр информации о жилье доступно для пользователя и гостя;
- создание бронирования доступно для пользователя;
- просмотр статуса и истории бронирований доступно для пользователя.

Коммуникация:

- ведение чата с владельцем доступно для пользователя;
- ведение чата с пользователем доступно для владельца.

Работа с отзывами:

- оставление отзыва и оценки после аренды доступно для пользователя.

Управление профилем:

- редактирование личных данных доступно для пользователя.

Эти прецеденты отражают ключевые взаимодействия пользователей с системой, обеспечивая четкое распределение функционала между ролями и способствуя удобству использования приложения.

2.3 Общая архитектура приложения

На рисунке 2.2 и в приложении Б представлена архитектурная схема веб-приложения в целом. Как можно увидеть, разрабатываемое веб-приложение состоит из трех частей: база данных, серверная часть и клиентская часть.

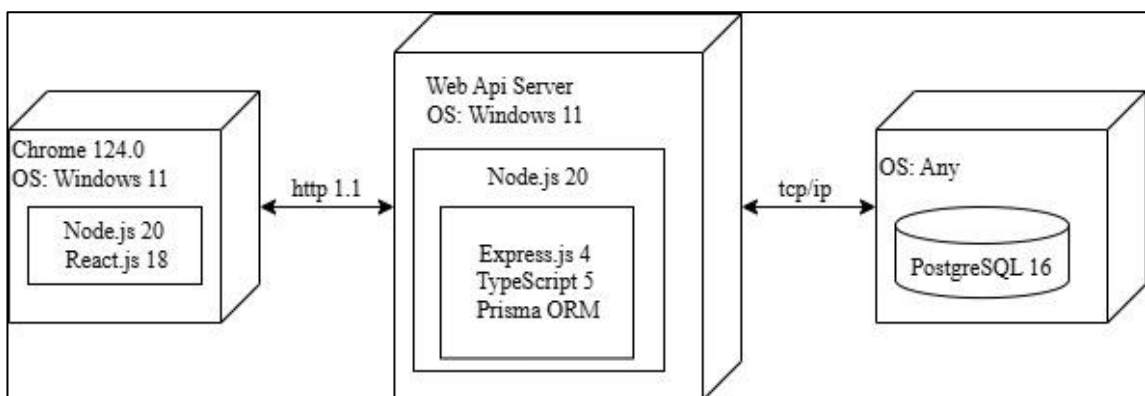


Рисунок 2.2 – Архитектурная схема веб-приложения

База данных приложения RentHouse размещается на сервере PostgreSQL 16. Серверная часть приложения написана на языке программирования TypeScript с использованием среды выполнения Node.js 20, фреймворка Express.js 4 и ORM Prisma, обеспечивая строгую типизацию и надежность кода. Для передачи данных между клиентом и сервером используется

протокол TCP, гарантирующий стабильное и последовательное соединение. Для запуска серверной части используется операционная система Windows 11. Клиентская часть запускается на операционной системе Windows 11 в браузере Google Chrome и написана с использованием библиотеки React.js 18, которая работает на основе Node.js.

2.4 Проектирование базы данных

Диаграмма базы данных таблиц (Database Table Diagram) – это визуальное представление структуры базы данных и отношений между таблицами, которые хранятся в этой базе данных. Проектирование базы данных является важным этапом разработки приложения, поскольку оно определяет, как данные будут организованы, храниться и обрабатываться. Грамотно спроектированная база данных обеспечивает эффективность хранения информации, быстрый доступ к данным, целостность и масштабируемость системы. Оно позволяет минимизировать избыточность данных, упростить выполнение запросов и обеспечить надежность работы приложения. Диаграмма представлена на рисунке 2.3, а также в приложении В.

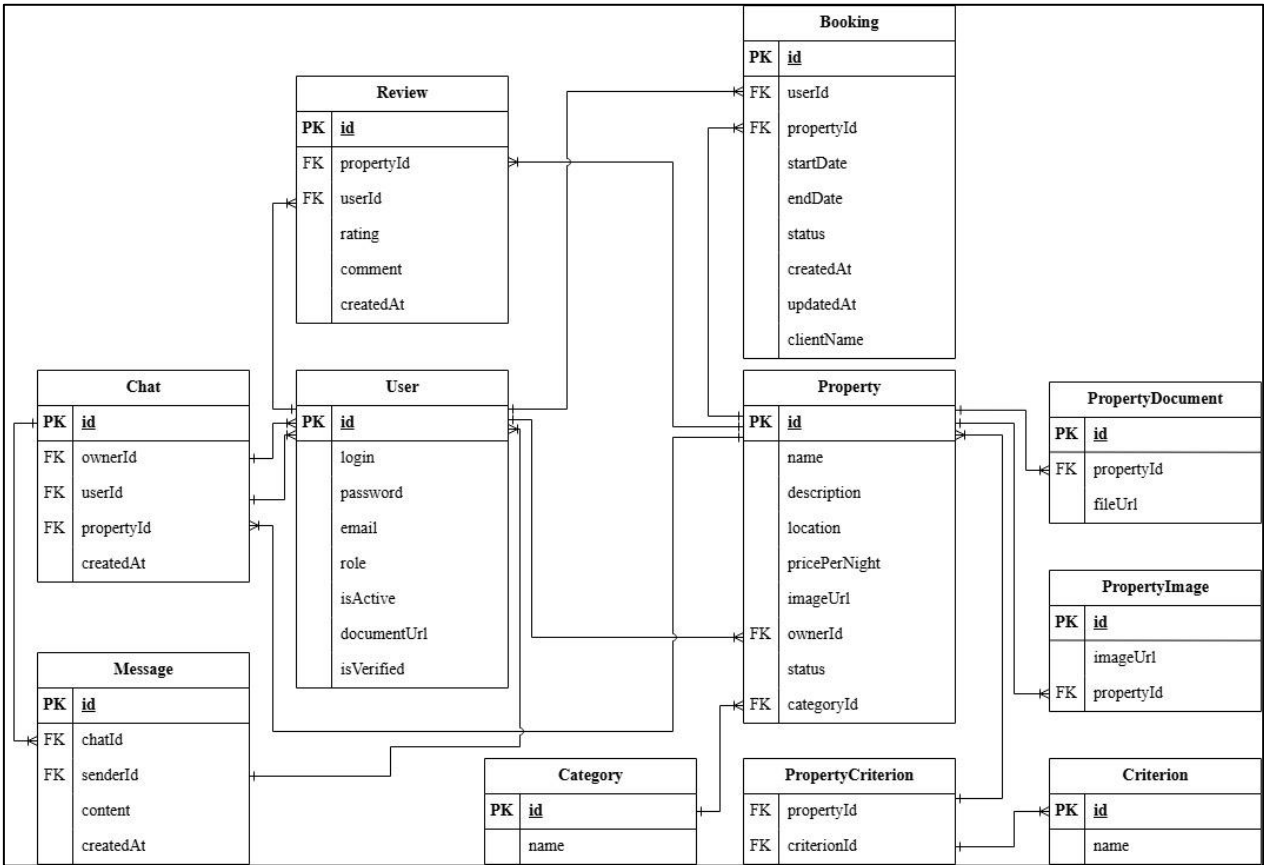


Рисунок 2.3 – Логическая схема базы данных

В этой базе данных используется сложная структура для управления пользователями, недвижимостью, бронированиями и отзывами. Модели и таблицы в базе данных связаны между собой с помощью различных связей и перечислений, что позволяет эффективно хранить и извлекать данные. Для

обеспечения функциональности приложения база данных включает следующие таблицы, каждая из которых выполняет определенную роль в системе:

- таблица User хранит информацию о пользователях системы, включая логин, пароль, email, роль (пользователь, владелец или администратор), ссылку на документ для верификации, а также статус активности и верификации;
- таблица Property содержит данные об объектах недвижимости, такие как название, описание, местоположение, цена за ночь, связь с владельцем и категорией, ссылка на документ, а также статус объекта;
- таблица PropertyDocument хранит ссылки на документы, связанные с объектами недвижимости, для подтверждения их легитимности;
- таблица Review содержит отзывы и оценки пользователей о жилье, включая рейтинг, комментарий и связь с пользователем и объектом недвижимости;
- таблица Booking хранит информацию о бронированиях, даты начала и окончания, статус бронирования, а также связь с пользователем и жильем;
- таблица Category содержит категории жилья (например, квартира, дом, апартаменты) для классификации объектов недвижимости;
- таблица Criterion хранит критерии жилья (например, наличие Wi-Fi, парковки) для фильтрации объектов недвижимости;
- таблица PropertyCriterion промежуточная таблица, связывающая объекты недвижимости с критериями отношение многие-ко-многим;
- таблица Chat хранит данные о чатах между пользователями и владельцами, включая связь с объектом недвижимости и участниками переписки;
- таблица Message содержит сообщения, отправленные в чатах, с указанием отправителя, содержимого и времени отправки;
- таблица PropertyImage хранит ссылки на изображения объектов недвижимости для их визуального представления.

Эти таблицы обеспечивают структурированное хранение данных и поддерживают функциональность приложения, включая поиск, бронирование, управление объектами и коммуникацию между пользователями.

Таблица User содержит существующих пользователей.

Таблица 2.1 – Структура таблицы «User»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор пользователя
login	TEXT	Not null	Логин пользователя
password	TEXT	Not null	Пароль пользователя
Email	TEXT	Not null	Адрес электронной почты
Role	TEXT	Not null	Роль пользователя
isActive	BOOLEAN	Not null	Статус пользователя
documentUrl	TEXT	Nullable	Прикрепленный документ пользователя

Продолжение таблицы 2.1

Название	Тип	Ограничения	Описание
isVerified	BOOLEAN	Not null	Статус верификации документа
created_at	DATETIME	Not null	Дата и время создания пользователя
updated_at	DATETIME	Not null	Дата и время последнего обновления

Столбец isActive отвечает за статус пользователя, он может быть заблокирован или нет. Столбец isVerified отвечает за то, верифицирован пользователь или нет, то есть сможет ли он стать владельцем или нет. Столбец role может принимать значение ADMIN, OWNER и USER.

Таблица Property содержит информацию о жилье.

Таблица 2.2 – Структура таблицы «Property»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор жилья
name	TEXT	Not null	Название жилья
description	TEXT	Not null	Описание жилья
location	TEXT	Not null	Адрес жилья
pricePerNight	INTEGER	Not null	Стоимость аренды за ночь
ownerId	INTEGER	Foreign key	Владелец жилья
categoryId	INTEGER	Foreign key	Категория жилья
status	TEXT	Not null	Статус жилья

Столбец ownerId служит для связи с таблицей User многие к одному. Столбец categoryId служит для связи с таблицей Category многие к одному. Столбец status может принимать значения PENDING, APPROVED и REJECTED, по умолчанию столбец имеет значение PENDING, статус меняет администратор сервиса.

Таблица PropertyDocument содержит документы для жилья.

Таблица 2.3 – Структура таблицы «PropertyDocument»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор документа
propertyId	INTEGER	Foreign key	Идентификатор жилья
fileUrl	TEXT	Not null	Загруженный файл

Столбец propertyId служит для связи с таблицей Property многие к одному. Столбец fileUrl служит для хранения документов, подтверждающих право собственности на жилье.

Таблица Review хранит отзывы жилья.

Таблица 2.4 – Структура таблицы «Review»

Название	Тип	Ограничения	Описание
Id	INTEGER	Primary key	Идентификатор отзыва
propertyId	INTEGER	Foreign key	Идентификатор жилья
userId	INTEGER	Foreign key	Идентификатор пользователя
rating	INTEGER	Not null	Оценка объект
comment	TEXT	Nullable	Текстовый комментарий к отзыву
createdAt	DATETIME	Not null	Дата и время создания отзыва

Столбец propertyId служит для с связи с таблицей Property многие к одному. Столбец userId служит для с связи с таблицей User многие к одному.

Таблица Booking хранит информацию о бронировании жилья.

Таблица 2.5 – Структура таблицы «Booking»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор бронирования
propertyId	INTEGER	Foreign key	Идентификатор жилья
userId	INTEGER	Foreign key	Идентификатор пользователя
startDate	DATETIME	Not null	Дата начала бронирования
endDate	DATETIME	Not null	Дата окончания бронирования
status	TEXT	Not null	Статус бронирования
createdAt	DATETIME	Not null	Дата и время создания бронирования
updatedAt	DATETIME	Not null	Дата и время обновления записи
clientName	TEXT	Not null	Имя пользователя

Столбец propertyId служит для с связи с таблицей Property многие к одному. Столбец userId служит для с связи с таблицей User многие к одному. Столбец status может принимать значения PENDING, CONFIRMED и CANCELLED, статус меняет владелец жилья.

Таблица Category содержит категории жилья.

Таблица 2.6 – Структура таблицы «Category»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор категории
name	TEXT	Not null	Название категории

Столбец name содержит наименования категорий жилья.

Таблица Criterion содержит критерии жилья.

Таблица 2.7 – Структура таблицы «Criterion»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор критерия
name	TEXT	Not null	Название критерия

Столбец name содержит наименования критериев жилья.

Таблица PropertyCriterion является промежуточной между такими таблицами как Property и Criterion.

Таблица 2.8 – Структура таблицы «PropertyCriterion»

Название	Тип	Ограничения	Описание
propertyId	INTEGER	Foreign key	Идентификатор жилья
criterionId	INTEGER	Foreign key	Идентификатор критерия

Столбцы propertyId и criterionId являются внешними ключами промежуточной таблицы, для связи таблиц «Property» и «Criterion».

Таблица Chat содержит информацию о чатах.

Таблица 2.9 – Структура таблицы «Chat»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор чата
ownerId	INTEGER	Foreign key	Идентификатор владельца жилья
userId	INTEGER	Foreign key	Идентификатор пользователя
propertyId	INTEGER	Foreign key	Идентификатор жилья
createdAt	DATETIME	Not null	Дата и время создания чата

Столбец ownerId служит для связи с таблицей User многие к одному. Столбец userId служит для связи с таблицей User многие к одному. Столбец propertyId служит для связи с таблицей Property многие к одному.

Таблица Message содержит информацию о сообщениях.

Таблица 2.10 – Структура таблицы «Message»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор сообщения
chatId	INTEGER	Foreign key	Идентификатор чата
senderId	INTEGER	Foreign key	Идентификатор отправителя

Продолжение таблицы 2.10

Название	Тип	Ограничения	Описание
content	TEXT	Not null	Текст сообщения
createdAt	DATETIME	Not null	Время отправки сообщения

Столбец chatId служит для связи с таблицей Chat многие к одному. Столбец senderId служит для связи с таблицей User многие к одному.

Таблица PropertyImage содержит информацию о чатах.

Таблица 2.11 – Структура таблицы «PropertyImage»

Название	Тип	Ограничения	Описание
id	INTEGER	Primary key	Идентификатор изображения
imageUrl	TEXT	Not null	Ссылка на изображение жилья
propertyId	INTEGER	Foreign key	Идентификатор жилья

Столбец propertyId служит для связи с таблицей Property многие к одному.

2.5 Проектирование основных алгоритмов

В данном разделе будут подробно рассмотрены алгоритмы, разработанные для реализации функциональности приложения, соответствующего целям и задачам дипломного проекта. Эти алгоритмы являются ключевыми элементами программной логики, обеспечивающей достижение поставленных целей, и представляют собой последовательность шагов, необходимых для выполнения конкретных функций приложения. Разработка алгоритмов проводилась с учетом требований к производительности, надежности и удобству использования конечного продукта.

Для визуализации и упрощения восприятия каждого алгоритма были созданы соответствующие блок-схемы. Блок-схемы являются важным инструментом в проектировании программного обеспечения, так как они предоставляют графическое представление алгоритмических процессов. Каждая блок-схема состоит из набора элементов, где каждый блок обозначает определенное действие, условие или событие, а стрелки между блоками указывают на последовательность выполнения операций. Такой подход позволяет не только упростить понимание логики работы алгоритма, но и облегчить процесс его анализа, оптимизации и документирования. Кроме того, использование блок-схем способствует выявлению потенциальных ошибок на этапе проектирования, что снижает вероятность их появления в программном коде.

Особенностью разработанных блок-схем является их универсальность и адаптивность. Они могут быть использованы как разработчиками для реализации кода, так и другими участниками проекта, включая тестировщиков и аналитиков, для понимания общей структуры работы приложения. Каждая блок-

схема была тщательно проработана с учетом всех возможных сценариев использования, что обеспечивает их полноту и достоверность.

Всего в ходе выполнения дипломного проекта было создано два алгоритма, каждый из которых будет описан ниже в соответствующем подразделе.

2.5.1 Алгоритм добавления жилья

Для того чтобы создать жилье в сервисе нужно проделать определенные действия, которые формируют определенный алгоритм.

Данный алгоритм представлен далее на рисунке 2.4 и в приложении Г.

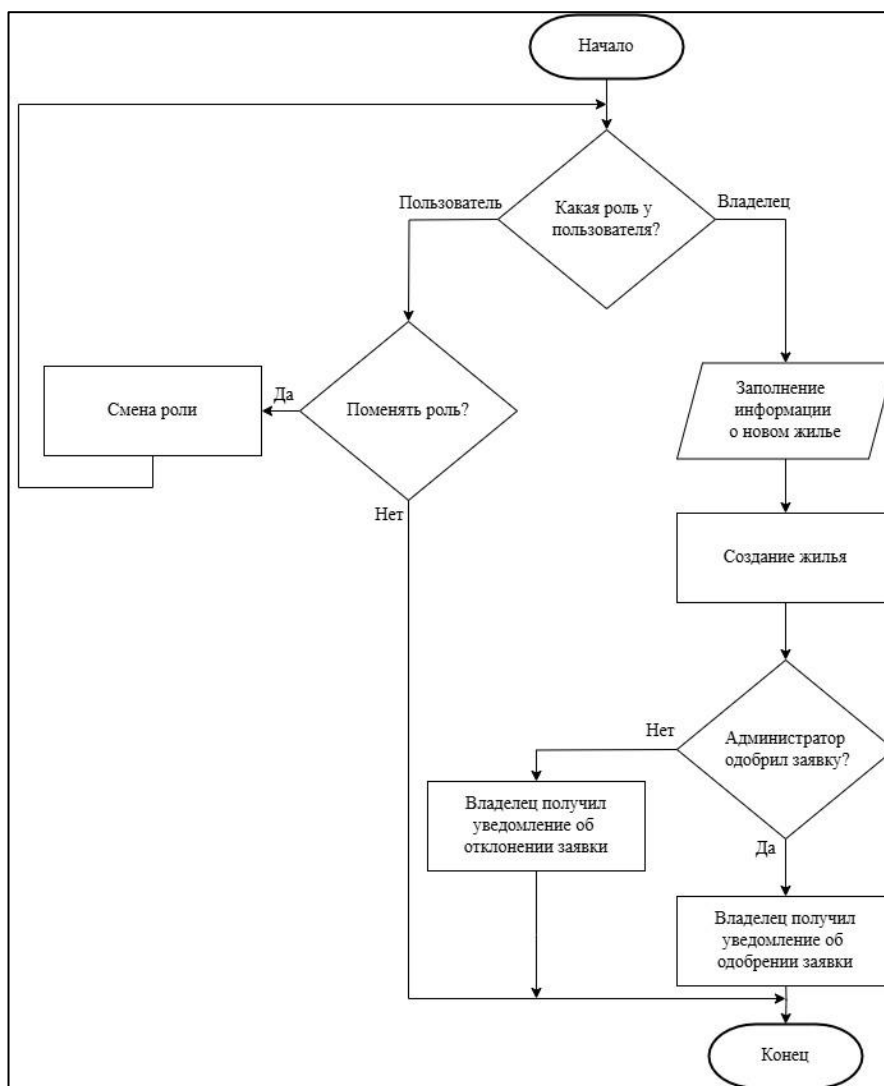


Рисунок 2.4 – Алгоритм добавления жилья

Для того чтобы создать жилье, у пользователя должна быть роль владельца, поэтому сервис сначала проверяет роль пользователя. Если у пользователя роль обычного пользователя, необходимо изменить ее на роль владельца. После смены роли появится возможность создания жилья, где нужно будет ввести определенные данные. После их ввода жилье добавится в каталог владельца, но для того, чтобы оно стало доступным для всех пользователей сервиса, оно должно пройти проверку администратором. Администратор

рассматривает документы, приложенные к жилью во время его создания, и принимает решение: одобрить жилье или отклонить. В случае отклонения жилье не будет отображаться другим пользователям, а останется видимым только владельцу. Если же администратор одобрил жилье, процесс создания жилья в сервисе считается успешно завершенным.

2.5.2 Алгоритм бронирования жилья

При попытке забронировать жилье на сервисе происходят определенный ряд действий, который формирует конкретный алгоритм.

Данный алгоритм представлен ниже на рисунке 2.5 и в приложении Д.

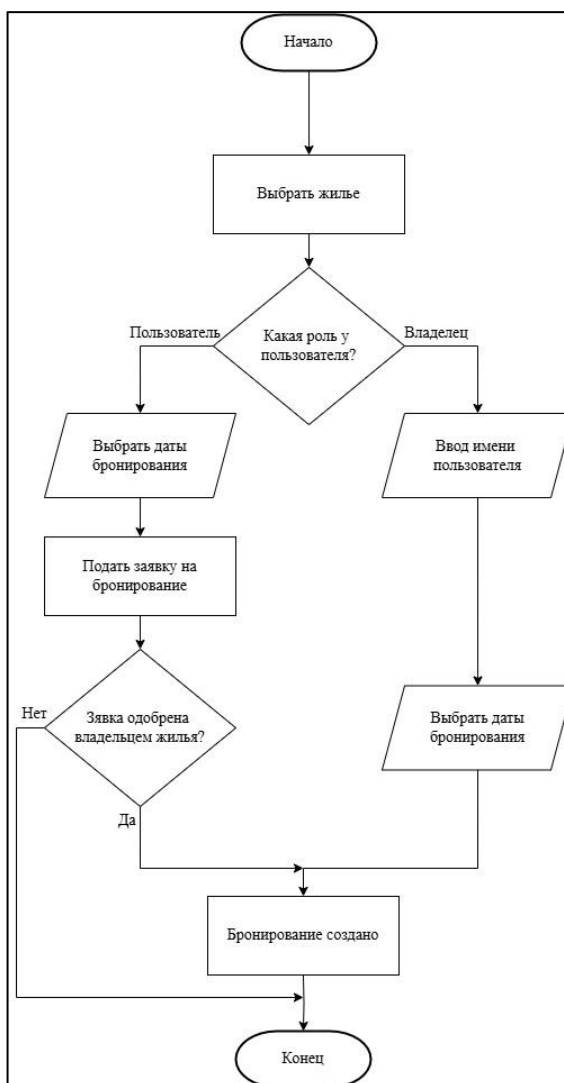


Рисунок 2.5 – Алгоритм бронирования жилья

Итак, при создании бронирования сначала проверяется роль пользователя. Если это обычный пользователь, нужно выбрать жилье, которое вы хотите забронировать, указать даты и подать заявку на бронирование. После этого владелец принимает решение: одобрить или отклонить заявку. В случае одобрения бронирование будет создано. Если же роль пользователя – владелец, сначала выбирается жилье, затем вводятся данные о пользователе,

создается пользователь, после чего указываются даты для бронирования. В результате создается бронирование. Таким образом, сервис предоставляет возможность создавать бронирования как для обычного пользователя, так и для владельца жилья, обеспечивая гибкость процесса.

2.6 Выводы по разделу

В разделе проектирования была построена и описана диаграмма вариантов использования. Эта диаграмма отражает основные функциональные требования к системе и демонстрирует типичные взаимодействия между пользователями и программным обеспечением, обеспечивая наглядное представление функционала.

В рамках данного раздела был произведен выбор технологий, языков программирования и инструментов, необходимых для реализации дипломного проекта. Выбор обоснован с точки зрения надежности, производительности, расширяемости и соответствия целям проекта.

Для реализации проекта были использованы следующие программные платформы, технологии и инструменты:

- Node.js 20 – как серверная платформа для обработки запросов и управления бизнес-логикой;
- TypeScript 5 – как основной язык программирования с поддержкой типизации и улучшенной читаемостью кода;
- Express.js 4 – как серверный фреймворк для построения REST API;
- React.js 18 – как библиотека создания удобного и интуитивно понятного пользовательского интерфейса;
- PostgreSQL 16 – как система управления базами данных, обеспечивающая реляционное хранение информации;
- Prisma ORM – как инструмент для работы с базой данных, предоставляющий удобный слой абстракции над SQL и поддержку миграций.

В проекте была выбрана реляционная СУБД PostgreSQL. На ее основе спроектирована и реализована структура базы данных, включающая 11 таблиц, каждая из которых отвечает за хранение определенных сущностей и связей между ними. Были реализованы ограничения целостности данных, а также связи между таблицами, обеспечивающие корректную обработку и валидацию информации при взаимодействии с приложением для надежной работы системы.

Также в рамках проектирования были разработаны два ключевых алгоритма, отражающие основную бизнес-логику приложения:

- алгоритм добавления жилья – описывает последовательность действий при создании нового объекта недвижимости, включая валидацию данных, загрузку изображений и прикрепление документов;
- алгоритм бронирования жилья – реализует процесс создания бронирования пользователем, включая проверку доступности жилья на выбранные даты и сохранение информации о брони в базе данных.

Каждый из этих алгоритмов был подробно описан и визуализирован в виде блок-схем, что способствует их последующей реализации и тестированию.

3 Разработка веб-приложения

Главной задачей данного дипломного проекта является создание веб-сервиса для аренды жилья. На основе анализа существующих аналогичных сервисов были сформулированы задачи: разработка программных компонентов для работы пользователей, обеспечение удобного и интуитивного интерфейса, а также соблюдение всех современных требований к выбору технологий и средств реализации. В последующих разделах будут подробно рассмотрены процессы создания базы данных, серверной и клиентской частей приложения.

3.1 Разработка серверной части

Серверная часть приложения была разработана с использованием современной платформы Node.js и языка программирования TypeScript, что обеспечивает высокую производительность, масштабируемость и строгую типизацию кода. Node.js, благодаря своей асинхронной архитектуре, позволяет эффективно обрабатывать большое количество одновременных запросов, что делает ее идеальным выбором для серверной части приложения. Использование TypeScript, в свою очередь, повышает надежность кода за счет статической типизации, что упрощает процесс отладки и предотвращает множество ошибок на этапе разработки. Такой подход способствует созданию более устойчивого и поддерживаемого программного продукта.

Основным файлом проекта является index.ts, который выполняет роль точки входа в приложение. В этом файле осуществляется инициализация всех необходимых модулей, включая библиотеки для работы с HTTP-запросами, настройки окружения и подключение к базе данных. Кроме того, в index.ts настраиваются маршруты приложения, которые определяют, как сервер будет обрабатывать входящие запросы от клиента. Маршруты организованы таким образом, чтобы обеспечить четкое разделение ответственности: каждый маршрут направляет запросы к соответствующему контроллеру, который выполняет необходимую бизнес-логику и формирует ответ для клиента. Такая структура способствует модульности и упрощает дальнейшее расширение функциональности приложения.

Для работы с базой данных используется Prisma, который служит эффективным ORM-инструментом для взаимодействия с PostgreSQL. Prisma значительно упрощает выполнение запросов и манипуляцию данными, позволяя работать с базой через удобный API. На рисунке 3.1 представлена схема структуры данного проекта, которая наглядно иллюстрирует организацию данных и взаимодействие между основными компонентами серверной части.

					ДП 03.00.ПЗ		
		ФИО	Подпись	Дата	3 Разработка веб-приложения		
Разраб.		Синяк П.С.					
Пров.		Комкова А.В.					
Н. контр.		Алешаускас В.А.					
Утв.		Блинова Е.А.					
					Лит.	Лист	Листов
					У	1	19
					БГТУ 1-40 05 01, 2025		

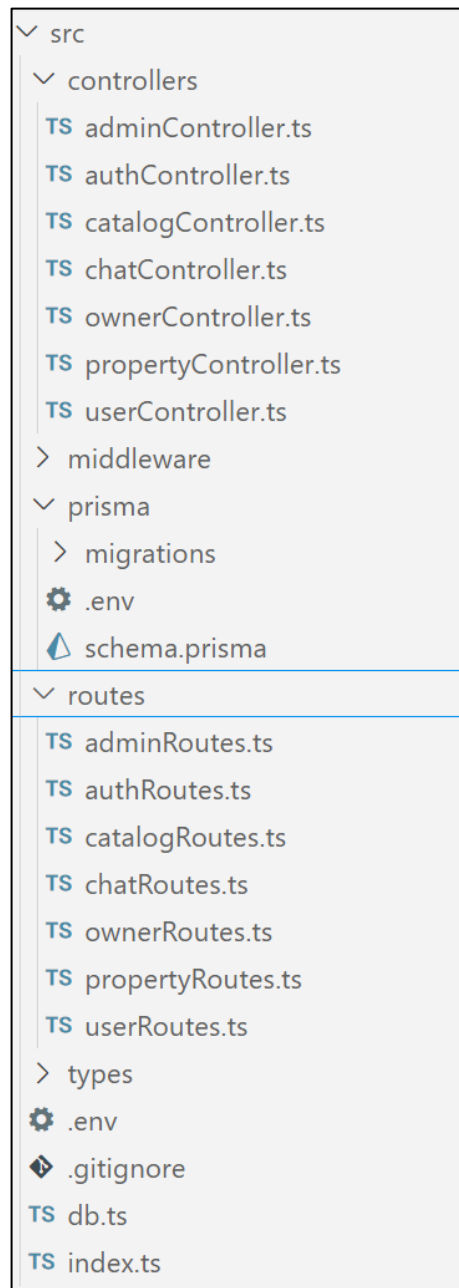


Рисунок 3.1 – Структура проекта

В приложении основная роль в обработке запросов принадлежит контроллерам и маршрутам. Контроллеры отвечают за обработку входящих HTTP-запросов, а маршруты направляют эти запросы на соответствующие контроллеры для выполнения действий, таких как создание, обновление или удаление данных. Контроллеры выполняют логику обработки запросов и вызывают соответствующие функции для работы с данными, что позволяет организовать код более структурированно.

Каждый контроллер связан с определенным функционалом приложения. Например, в проекте контроллер `catalogController` занимается созданием жилья с загрузкой изображений, а другие контроллеры, такие как `authRoutes`, `userRoutes`, `propertyRoutes` и `adminRoutes`, обрабатывают различные функциональные блоки, такие как авторизация, работа с пользователями и администрирование.

Маршруты (routes) служат связующим звеном между входящими запросами и контроллерами. В отличие от контроллеров, маршруты не содержат бизнес-логику, а просто перенаправляют запросы на соответствующие функции в контроллерах. В коде маршруты подключаются в файле `index.ts`, который является точкой входа в приложение и обеспечивает его запуск. На основе структуры проекта выделены следующие маршруты с их назначением:

- файл `adminRoutes.ts` содержит маршруты для управления административными функциями, такими как редактирование категорий, критериев жилья и управление доступом пользователей и их статусом;
- файл `authRoutes.ts` содержит маршруты, связанные с регистрацией и авторизацией пользователей, включая создание учетной записи и вход в систему;
- файл `catalogRoutes.ts` содержит маршруты для работы с каталогом жилья, включая поиск, фильтрацию и просмотр информации об объектах недвижимости;
- файл `chatRoutes.ts` содержит маршруты для обработки чатов между пользователями и владельцами, включая отправку и получение сообщений;
- файл `ownerRoutes.ts` содержит маршруты для владельцев недвижимости, такие как добавление, редактирование и удаление объектов аренды, а также управление бронированиями и просмотр статистики владельца;
- файл `propertyRoutes.ts` содержит маршруты для работы с объектами недвижимости, просмотр деталей, загрузку документов и управление статусами;
- файл `userRoutes.ts` содержит маршруты, связанными с профилем пользователя, такими как просмотр истории бронирований, редактирование данных и оставление отзывов, а также изменение настроек пользователя;

Эти маршруты обеспечивают четкое разделение функциональности и упрощают взаимодействие между клиентской и серверной частями приложения. Каждый маршрут был спроектирован с учетом принципов RESTful API, что обеспечивает стандартизированный подход к обработке запросов и повышает совместимость с клиентскими приложениями. Кроме того, структура маршрутов позволяет легко масштабировать приложение, добавляя новые функциональные возможности без необходимости переработки существующего кода.

Файл `index.ts` является главным файлом сервера, где настраиваются маршруты и запускается сервер. В этом файле создается экземпляр приложения Express, подключаются все маршруты через `app.use()`, а также настраиваются middleware для обработки запросов, таких как CORS, парсинг JSON и загрузка файлов через `multer`. В конце происходит запуск сервера и подключение к базе данных через Prisma. Таким образом, `index.ts` является основным файлом, где происходит настройка всего приложения, подключение всех необходимых компонентов и их взаимодействие.

Структура проекта организована таким образом, чтобы контроллеры и маршруты могли легко взаимодействовать друг с другом, при этом каждая часть приложения отвечает только за свою область ответственности. Такой подход к организации кода обеспечивает высокую степень модульности, что упрощает тестирование и дальнейшее развитие приложения. Кроме того, четкое разделение ответственности между маршрутами и контроллерами позволяет разработчикам быстро ориентироваться в проекте и вносить изменения

без риска нарушения общей функциональности системы. Файл `index.ts` представлен в листинге 3.1 и служит точкой входа для инициализации системы.

```
const app = express();
const PORT = process.env.PORT || 3000;
app.use(cors({
  origin: 'http://localhost:3001',
}));
app.use(express.json());
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'images/');
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' +
Math.round(Math.random() * 1e9);
    cb(null, file.fieldname + '-' + uniqueSuffix + path.ex-
tname(file.originalname));
  },
});
const upload = multer({ storage });
app.post('/housing', upload.single('image'), createHousing);
app.use('/images', express.static(path.join(__dirname, '../im-
ages')));
app.use('/auth', authRoutes);
app.use('/users', userRoutes);
app.use('/catalog', catalogRoutes);
app.use('/property', propertyRoutes);
app.use('/admin', adminRoutes);
app.listen(PORT, async () => {
  await prisma.$connect();
  console.log(`Сервер запущен на http://localhost:${PORT}`);
});
```

Листинг 3.1 – Файл `index.ts`

Файл `tsconfig.json` в проекте на TypeScript играет ключевую роль в настройке компилятора и определении того, как будет происходить преобразование TypeScript в JavaScript. Этот файл является центральным элементом конфигурации для любого проекта, использующего TypeScript, так как он задает правила и параметры, которые определяют поведение компилятора, структуру выходного кода и совместимость с другими инструментами разработки. В контексте данного проекта, который построен с использованием платформы Node.js и фреймворка Express, настройки в `tsconfig.json` были тщательно подобраны для обеспечения корректной работы приложения, его производительности и удобства разработки.

В данном проекте файл `tsconfig.json` был тщательно протестирован, чтобы убедиться, что все параметры работают корректно и не вызывают конфликтов. Например, проверялось, что скомпилированный код запускается без ошибок в среде Node.js, а также что интеграция с Express и Prisma происходит

без проблем. Также проводились проверки на предмет избыточных настроек, чтобы минимизировать время компиляции и упростить поддержку конфигурации. Данный файл представлен в листинге 3.2, где подробно описаны все используемые параметры и их назначение.

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "commonjs",
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "outDir": "./dist",
    "typeRoots": [
      "./node_modules/@types",
      "./src/types"
    ],
    "types": ["node", "express"]
  },
  "include": [
    "src/**/*.ts",
    "src/**/*.tsx"
  ],
  "exclude": [
    "node_modules"
  ]
}
```

Листинг 3.2 – Файл tsconfig.json

Параметр `target`: устанавливает версию ECMAScript, в которую будет компилироваться TypeScript. В вашем случае это ES6, что означает, что компилятор будет генерировать код, соответствующий стандарту ECMAScript 2015 (ES6), включая поддержку классов, стрелочных функций, промисов и других функций этого стандарта.

Параметр `module`: указывает, какой модульный формат будет использоваться. В вашем случае это `commonjs`, который является стандартом для Node.js. Это означает, что компилятор будет генерировать код с использованием `require()` и `module.exports` для импорта и экспорта модулей.

Параметр `esModuleInterop`: включает поддержку взаимодействия с модулями, экспортируемыми по стандарту ES, и позволяет использовать их в стиле CommonJS. Это упрощает работу с модулями, которые используют разные стандарты, и позволяет делать импорты в стиле `import * as X from 'module'`.

Параметр `strict`: включает строгий режим, который активирует множество проверок типов, таких как `noImplicitAny`, `strictNullChecks` и другие, что помогает предотвратить потенциальные ошибки и улучшить качество кода.

Параметр `skipLibCheck`: Отключает проверку типов в файлах `.d.ts` (определения типов), что может ускорить компиляцию, особенно в больших проектах с множеством зависимостей и сложной структурой.

Параметр `forceConsistentCasingInFileNames`: устанавливает требование использовать согласованный регистр в именах файлов, предотвращая проблемы с кроссплатформенной совместимостью.

Параметр `outDir`: указывает, куда будет скомпилирован результат работы компилятора. В вашем случае это папка `./dist`, куда будут выводиться все скомпилированные `.js` файлы после завершения процесса.

Параметр `typeRoots`: определяет директории, в которых компилятор будет искать типы. В вашем случае компилятор будет искать типы сначала в папке `node_modules/@types` (стандартные типы, установленные через `npm`), а затем в `src/types` (пользовательские типы, которые вы можете создать).

Параметр `types`: определяет, какие типы следует подключить. В вашем случае это типы для `Node.js` и `Express`, что позволяет использовать их в проекте для авто дополнения и проверки типов с высокой точностью.

В рамках данного проекта будут рассматриваться следующие функции:

- регистрация;
- авторизация;
- управление жильем в системе;
- бронирование жилья;
- оставление отзывов с оцениванием после сделки.

3.1.1 Регистрация

В любом приложении одной из ключевых задач является правильная и безопасная регистрация и авторизация пользователей. Эти процессы должны быть спроектированы таким образом, чтобы гарантировать защиту данных пользователей, обеспечивать удобство использования и беспрепятственную интеграцию с другими частями системы. Регистрация и авторизация являются основой для управления доступом пользователей к функциональности приложения, а их надежность напрямую влияет на доверие к системе и общий пользовательский опыт. В данном разделе будет рассмотрен процесс регистрации пользователей, включая его техническую реализацию, меры безопасности и особенности интеграции с другими компонентами приложения.

Для реализации регистрации в приложении используется контроллер `authController`, который управляет обработкой запроса на регистрацию. Этот контроллер выполняет центральную роль в обработке входящих данных, их валидации и сохранении в базе данных. В запросе от клиента передаются такие данные, как `email`, пароль и роль пользователя, при этом роль пользователя указывается на стороне клиента. Эти данные проходят тщательную проверку на сервере, чтобы обеспечить их корректность и соответствие заданным требованиям. Например, `email` проверяется на уникальность и правильность формата, а пароль – на соответствие минимальным требованиям безопасности, таким как длина и наличие определенных символов. Код, реализующий логику регистрации, будет представлен в листинге 3.3.

```

export const register = async (req: Request, res: Response):
Promise<void> => {
  const { login, password, email } = req.body;
  if (existingUser) {
    res.status(400).json({ message: 'Пользователь с таким ло-
гином или email уже существует.' });
    return;
  }
  const hashedPassword = await bcrypt.hash(password, 10);
  const newUser = await prisma.user.create({
    data: {
      login,
      password: hashedPassword,
      email,
      role: 'USER',
    },
  });
  const token = jwt.sign(
    { userId: newUser.id, role: newUser.role },
    JWT_SECRET,
    { expiresIn: '72h' }
  );
  res.status(201).json({
    message: 'Пользователь успешно зарегистрирован',
    token,
  });
}
};

```

Листинг 3.3 – Код регистрации

Код представляет собой обработчик маршрута для регистрации пользователя в приложении. Когда клиент отправляет запрос на регистрацию с данными пользователя, такими как логин, пароль и email, сервер выполняет несколько шагов для создания нового пользователя. Сначала данные из запроса извлекаются, и проверяется, существует ли уже пользователь с таким логином или email в базе данных.

Если такой пользователь найден, сервер возвращает ошибку, уведомляя клиента, что логин или email уже заняты. Если же такого пользователя нет, сервер переходит к следующему шагу – хешированию пароля. Для этого используется библиотека `bcrypt`, которая безопасно преобразует исходный пароль в хеш, защищая его от несанкционированного доступа. После успешного хеширования пароля создается новый пользователь в базе данных с предоставленными данными, при этом роль пользователя по умолчанию устанавливается как «USER».

В этот же момент генерируется JWT (JSON Web Token), который содержит идентификатор пользователя, его роль и срок действия. Токен остается действительным в течение 72 часов, обеспечивая временный доступ к защищенным ресурсам приложения. Если процесс регистрации проходит успешно, сервер отправляет клиенту ответ с сообщением об успехе и сгенерированным токеном. В

случае возникновения ошибок (например, проблем с базой данных или хешированием пароля), сервер возвращает ошибку с описанием проблемы.

3.1.2 Безопасность

Процесс регистрации обеспечивает надежную защиту данных пользователей и безопасность паролей, что критично для современных веб-приложений. Использование JWT токенов играет ключевую роль в управлении сессиями и аутентификации. JWT представляет собой компактный и самодостаточный способ передачи информации между сторонами, который закодирован и может быть подписан криптографически. Это позволяет гарантировать целостность данных и их происхождение, исключая возможность подделки.

Токены позволяют избежать необходимости хранения сессий на сервере, что снижает нагрузку и повышает масштабируемость приложения. Они передаются в заголовках HTTP-запросов, что упрощает аутентификацию на стороне клиента без необходимости многократного ввода учетных данных. Благодаря своей универсальности и поддержке стандартов, он интегрируется с большинством современных фреймворков и библиотек, таких как Express.js, а также обеспечивает гибкость в настройке сроков действия и обновления токенов.

Использование JWT в данном проекте обеспечивает удобное и безопасное управление доступом, позволяя проверять права пользователя на каждом запросе. Это особенно важно для системы аренды жилья, где требуется защита личных данных и управление доступом к объектам недвижимости. Таким образом, внедрение JWT не только повышает безопасность, но и оптимизирует работу приложения, делая его более устойчивым к атакам и удобным для пользователей.

Кроме того, использование JWT позволяет легко интегрировать систему аутентификации с различными внешними сервисами и платформами, предоставляя возможность для единой точки входа. Это может быть полезно для расширения функционала, например, через авторизацию через социальные сети или сторонние идентификационные провайдеры. JWT также позволяет реализовать механизмы автоматического продления сессий и безопасного завершения сеансов, что повышает как удобство, так и безопасность пользователей.

3.1.3 Авторизация

Авторизация – это ключевая часть безопасности приложения. Она предоставляет пользователям доступ к ограниченным ресурсам после успешной проверки их личности. Этот процесс включает подтверждение данных пользователя и создание сессии, что позволяет контролировать доступ к защищенным частям системы.

В данном случае процесс авторизации осуществляется через проверку учетных данных пользователя, таких как логин и пароль, с последующей выдачей токена для дальнейшего взаимодействия с системой. Логика авторизации обеспечит безопасность и управление правами пользователей в системе. Код, реализующий процесс авторизации, будет представлен в листинге 3.4.

```

export const login = async (req: Request, res: Response) :
Promise<void> => {
  const { login, password } = req.body;
  try {
    const user = await prisma.user.findUnique({
      where: { login },
    });
    if (!user) {
      res.status(400).json({ message: 'Неверный логин или па-
роль' });
      return;
    }
    const isPasswordValid = await bcrypt.compare(password,
user.password);
    if (!isPasswordValid) {
      res.status(400).json({ message: 'Неверный логин или па-
роль' });
      return;
    }
    const token = jwt.sign({ userId: user.id, role: user.role
}, JWT_SECRET, {
      expiresIn: '72h',
    });
    res.status(200).json({ message: 'Авторизация успешна', to-
ken });
  } catch (error) {
    res.status(500).json({ message: 'Ошибка при авторизации',
error });
  }
};

```

Листинг 3.4 – Код авторизации

Этот контроллер реализует процесс авторизации пользователя. Когда пользователь пытается войти в систему, отправляется запрос с логином и паролем. Контроллер извлекает эти данные из тела запроса и ищет пользователя в базе данных, используя логин. Если пользователь с таким логином не найден, сервер отправляет ошибку с сообщением, что логин или пароль неверные.

Если пользователь найден, следующим шагом идет проверка пароля. Для этого используется метод `bcrypt`, который сравнивает введенный пароль с хешированным паролем, хранящимся в базе данных. Если пароли не совпадают, также возвращается ошибка с аналогичным сообщением.

Если логин и пароль верны, генерируется JWT-токен, который содержит информацию о пользователе (его ID и роль). Этот токен используется для авторизации и доступа к защищенным частям приложения. Токен отправляется пользователю в ответе, и с его помощью он может выполнять дальнейшие запросы в систему без необходимости вводить логин и пароль повторно.

Если во время выполнения процесса авторизации возникает ошибка, например при работе с базой данных или другим компонентом, сервер возвращает сообщение об ошибке с соответствующим кодом состояния.

3.1.4 Управление жильем в системе

Управление жильем в системе – это процесс, который включает создание, изменение и удаление информации о жилье, доступном на платформе для бронирования. В этой части приложения пользователи (в частности, владельцы жилья) могут добавлять новое жилье в систему, редактировать его характеристики или удалять, если оно больше недоступно.

Управление жильем помогает поддерживать актуальность информации на платформе и дает возможность пользователям искать жилье по определенным критериям. В листинге 3.5 будет представлен код для создания жилья.

```
export const createHousing = async (req: Request, res: Response): Promise<void> => {
  const { name, description, location, pricePerNight, ownerId,
categoryId, criteria } = req.body;
  const imageUrl = req.file ? `/images/${req.file.filename}` :
null;
  if (!name || !description || !location || !pricePerNight ||
!ownerId || !categoryId) {
    res.status(400).json({ message: 'Все поля, кроме изображе-
ния, обязательны' });
    return;
  }
  try {
    const newProperty = await prisma.property.create({
      data: {
        name,
        description,
        location,
        pricePerNight: parseFloat(pricePerNight),
        owner: {
          connect: { id: parseInt(ownerId) },
        },
        category: {
          connect: { id: parseInt(categoryId) },
        },
        imageUrl,
      },
    });
  }
};
```

Листинг 3.5 – Код для создания жилья

Контроллер `createHousing` отвечает за процесс создания нового жилья в системе. Вначале он извлекает данные из тела запроса, такие как название жилья, описание, местоположение, цена за ночь, ID владельца, ID категории и критерии. Также, если изображение жилья было загружено, извлекается путь к файлу.

После этого контроллер проверяет, что все обязательные поля присутствуют в запросе. Если какое-то поле отсутствует (кроме изображения),

возвращается ошибка с кодом 400 и соответствующим сообщением. Это позволяет предотвратить создание записи с недостаточным набором данных.

Если все поля в порядке, контроллер использует Prisma для создания записи о жилье в базе данных. Он связывает новое жилье с владельцем и категорией через их ID, передаваемые в запросе. В случае наличия изображения сохраняется путь к этому изображению. Если в запросе также указаны критерии для жилья (например, наличие бассейна или Wi-Fi), контроллер связывает эти критерии с новым жильем через промежуточную таблицу, добавляя соответствующие записи в базу данных.

По завершении всех операций контроллер возвращает клиенту ответ с кодом 201, включая данные нового жилья. В случае ошибки (например, при взаимодействии с базой данных) контроллер перехватывает исключение и отправляет ошибку с кодом 500, информируя клиента о проблемах на сервере. Таким образом, контроллер эффективно управляет процессом добавления нового жилья в систему, обеспечивая валидацию данных и корректное взаимодействие с базой данных.

В листинге 3.6 будет представлен код для удаления жилья.

```
export const deleteHousing = async (req: Request, res: Response): Promise<void> => {
  try {
    const property = await prisma.property.findUnique({
      where: { id: parseInt(id) },
      include: { bookings: true, criteria: true
    },
  });
  await prisma.booking.deleteMany({
    where: { propertyId: parseInt(id),
  },
});
  await prisma.propertyCriterion.deleteMany({
    where: { propertyId: parseInt(id),
  },
});
  await prisma.property.delete({
    where: { id: parseInt(id) },
  });
}
};
```

Листинг 3.6 – Код для удаления жилья

Контроллер deleteHousing отвечает за удаление жилья из системы. Когда поступает запрос на удаление, контроллер извлекает ID жилья из параметров запроса. Также он получает данные о текущем пользователе, которые предположительно были добавлены в объект req через middleware, например, для проверки авторизации.

Контроллер разработан с учетом строгого соблюдения принципов безопасности и целостности данных. Перед выполнением любых операций

удаления он гарантирует, что все действия будут выполнены только после успешной проверки прав доступа. Такой подход обеспечивает высокую надежность даже при обработке больших объемов данных. Контроллер также поддерживает обработку ошибок на всех этапах, что исключает возможность частичного выполнения операций. Это делает его устойчивым к различным сбоям в системе.

На первом этапе контроллер проверяет, авторизован ли пользователь. Если данных о пользователе нет (например, он не прошел аутентификацию), возвращается ошибка с кодом 401 (неавторизован) и сообщением о необходимости авторизации. Затем контроллер пытается найти жилье в базе данных по ID, который был передан в запросе. Вместе с данными о жилье он также запрашивает связанные с ним бронирования и критерии (если они есть), чтобы удостовериться, что все связанные сущности будут корректно обработаны. Если жилье не найдено, возвращается ошибка с кодом 404 (не найдено), и процесс удаления не продолжается.

После того как жилье найдено, контроллер проверяет, имеет ли пользователь право на его удаление. Это право есть только у администратора или владельца данного жилья. Если пользователь не является владельцем жилья или администратором, возвращается ошибка с кодом 403 (Forbidden), информируя, что у пользователя нет прав на выполнение данного действия.

Если проверки пройдены успешно, контроллер сначала удаляет все бронирования, связанные с данным жильем, через метод `deleteMany`. Эта операция выполняется в рамках транзакции, чтобы обеспечить атомарность и согласованность данных. Такой подход минимизирует риск частичного удаления связанных сущностей. Затем он удаляет все связи этого жилья с критериями через таблицу `propertyCriterion`, которая является промежуточной. Эти операции необходимы, чтобы в базе данных не оставалось зависимых данных, которые могут нарушить ее целостность после удаления самого жилья.

После того как все зависимые сущности были удалены, контроллер удаляет само жилье через метод `delete` и отправляет пользователю ответ с кодом 200 (OK) и сообщением о том, что жилье и все связанные с ним данные были успешно удалены. Для повышения надежности процесса удаления используется механизм логирования всех операций. Это позволяет отслеживать успешные действия и быстро выявлять возможные сбои. Такой подход упрощает поддержку и аудит системы. Если в процессе удаления возникает ошибка, она перехватывается, и пользователю отправляется ответ с кодом 500 (Internal Server Error), а также логируется информация о произошедшей ошибке для дальнейшего анализа.

Таким образом, данный контроллер обеспечивает не только удаление самого жилья, но и гарантирует, что все связанные данные, такие как бронирования и критерии, будут корректно удалены, предотвращая проблемы с целостностью базы данных.

В листинге 3.7 будет представлен код для редактирования жилья.

```

export const updateProperty = async (req: Request, res: Response): Promise<void> => {
  try {
    const updatedProperty = await prisma.property.update({
      where: { id: Number(id) },
      data: {
        name,
        description,
        location,
        pricePerNight: parseFloat(pricePerNight),
        category: categoryId ? { connect: { id: parseInt(categoryId) } } : undefined,
        ...(imageUrl && { imageUrl }),
      },
    });
    if (validCriteria.length > 0) {
      await prisma.propertyCriterion.deleteMany({
        where: { propertyId: Number(id) },
      });
      const criterionConnections = validCriteria.map((criterionId: number) => ({
        propertyId: updatedProperty.id,
        criterionId,
      }));
      await prisma.propertyCriterion.createMany({
        data: criterionConnections,
      });
    }
  }
};

```

Листинг 3.7 – Код для редактирования жилья

Контроллер `updateProperty` отвечает за обновление информации о жилье. Сначала он извлекает ID жилья из параметров запроса и данные для обновления из тела запроса. Если переданы данные для загрузки нового изображения, контроллер сохраняет путь к файлу. Если передается критерий (`criteria`), контроллер проверяет, является ли он строкой или массивом. Если это строка, она преобразуется в массив с проверкой на корректность значений.

Затем контроллер пытается найти жилье в базе данных по ID. Если жилье не найдено, возвращается ошибка с кодом 404 (не найдено). Если жилье найдено, контроллер обновляет его данные в базе, включая основную информацию (название, описание, местоположение, цена, категория) и, при наличии, новое изображение.

Если были переданы корректные критерии, сначала удаляются старые связи этого жилья с критериями, а затем создаются новые, используя промежуточную таблицу для связи жилья и критериев.

Наконец, обновленные данные о жилье возвращаются в ответе с кодом 200 (ОК). Если возникает ошибка, она логируется, и пользователю отправляется сообщение об ошибке с кодом 500 (внутренняя ошибка сервера).

3.1.5 Бронирование жилья

Бронирование жилья – это процесс, позволяющий пользователям резервировать выбранное жилье на определенные даты. В системе бронирования важно учитывать доступность жилья на заданный период, чтобы избежать пересечений с уже подтвержденными бронированиями. Этот функционал обычно включает проверку на доступность, создание заявки на бронирование и управление статусами бронирований. Реализация бронирования будет показана в листинге 3.8.

```
export const createBooking = async (req: Request, res: Response): Promise<void> => {
  const { propertyId, userId, startDate, endDate } = req.body;
  try {
    const existingBookings = await prisma.booking.findMany({
      where: {
        propertyId: propertyId,
        status: 'CONFIRMED',
        OR: [{
          startDate: { gte: new Date(startDate) },
          endDate: { lte: new Date(endDate) }
        },
        {
          startDate: { lte: new Date(endDate) },
          endDate: { gte: new Date(startDate) }
        }
      ],
    });
  },
  });
  const newBooking = await prisma.booking.create({
    data: {
      userId,
      propertyId,
      startDate: new Date(startDate),
      endDate: new Date(endDate),
      status: 'PENDING',
    },
  });
};
```

Листинг 3.8 – Создание бронирования жилья

Контроллер `createBooking` используется для обработки запросов на создание бронирования жилья. Основная задача этого контроллера – гарантировать, что новое бронирование не пересекается с уже существующими, и создать новую запись о бронировании в базе данных.

Когда клиент отправляет запрос, содержащий `propertyId` (идентификатор жилья), `userId` (идентификатор пользователя), а также даты начала (`startDate`) и окончания (`endDate`) бронирования, контроллер начинает выполнение с проверки доступности жилья на указанный период. Первый шаг – это

поиск существующих подтвержденных бронирований для данного жилья, которые пересекаются с указанным диапазоном дат. Если хотя бы одно пересечение найдено, контроллер возвращает ответ с кодом состояния 400 и сообщает, что указанный период занят.

Если никаких пересечений не обнаружено, система переходит к созданию нового бронирования. Для этого в базу данных добавляется запись с переданными `userId`, `propertyId`, датами начала и окончания. Новое бронирование изначально получает статус `PENDING`, что означает, что оно ожидает подтверждения. Это может быть полезно, если система предполагает ручное или автоматическое подтверждение бронирований на следующем этапе.

В случае успешного создания бронирования контроллер возвращает ответ с кодом состояния 201, содержащий данные о созданной записи. Если в процессе выполнения возникает ошибка (например, из-за проблем с базой данных), контроллер обрабатывает ее и возвращает ответ с кодом состояния 500, указывая, что произошла ошибка на сервере. Этот подход обеспечивает целостность данных и предотвращает конфликты бронирований, сохраняя стабильность работы системы.

3.1.6 Оставление отзывов с оцениванием после сделки

Оставление отзывов и оценивание после сделки – это важный механизм, который позволяет пользователям делиться своим опытом взаимодействия с системой или другими пользователями. В контексте аренды жилья отзывы служат инструментом обратной связи, где арендаторы могут оценивать жилье, делиться своими впечатлениями о его состоянии, удобствах и соответствии ожиданиям. Оценки помогают новым пользователям принимать более осознанные решения, а также мотивируют владельцев жилья поддерживать высокий уровень сервиса. Логика работы этого функционала будет представлена в листинге 3.9.

```
export const addReview = async (req: Request, res: Response):
Promise<void> => {
  try {
    if (existingReview) {
      const newReview = await prisma.review.create({
        data: {
          propertyId: Number(propertyId),
          userId: Number(userId),
          rating,
          comment,
        },
        include: { user: true },
      });
      res.status(201).json(newReview);
    }
  }
};
```

Листинг 3.9 – Код для добавления отзыва

Контроллер `addReview` отвечает за добавление или обновление отзыва пользователя для конкретного жилья. Его задача – обработать данные, отправленные клиентом, проверить существование отзыва и в зависимости от результата либо создать новый, либо обновить уже существующий.

Когда запрос поступает, контроллер сначала извлекает из тела запроса следующие данные: идентификатор жилья (`propertyId`), идентификатор пользователя (`userId`), оценку (`rating`) и комментарий (`comment`). Эти данные нужны, чтобы связать отзыв с конкретным жильем и пользователем.

Затем контроллер проверяет, существует ли уже отзыв, оставленный этим пользователем для указанного жилья. Для этого в базе данных выполняется поиск отзыва по уникальной комбинации `propertyId` и `userId`. Если такой отзыв находится, он обновляется: заменяются старая оценка и комментарий на новые данные, предоставленные клиентом. После обновления контроллер возвращает обновленный отзыв с кодом успешного выполнения 200.

Если отзыв не найден, контроллер создает новый. Он записывает в базу данных все предоставленные данные, включая идентификатор жилья, пользователя, оценку и комментарий. Новый отзыв возвращается клиенту с кодом успешного создания 201.

На каждом этапе проверяются возможные ошибки. Если происходит сбой, например, при работе с базой данных, контроллер регистрирует ошибку в журнале и возвращает клиенту сообщение с кодом состояния 500, указывающим на ошибку сервера. Таким образом, этот контроллер реализует удобный механизм, который автоматически определяет, нужно ли обновить существующий отзыв или создать новый, упрощая взаимодействие пользователя с системой.

3.2 Разработка клиентской части

Клиентская часть веб-приложения была разработана с учетом принципов кроссплатформенности и работает в браузере. Все данные обновляются динамически, благодаря чему взаимодействие с интерфейсом становится плавным, быстрым и удобным.

Для обмена данными с сервером клиентская часть использует встроенный API `fetch`. Эта технология позволяет отправлять запросы к серверу и получать ответы в асинхронном режиме, что предотвращает задержки в работе интерфейса. Запросы отправляются с использованием различных HTTP-методов, таких как `GET`, `POST`, `PUT` и `DELETE`, на заранее определенные пути.

Подход с использованием `fetch` обеспечивает простоту реализации и достаточную гибкость для большинства задач. Кроме того, использование `fetch` позволяет эффективно обрабатывать ошибки и управлять заголовками запросов, что повышает надежность обмена данными. Все запросы оптимизированы для минимизации нагрузки на сервер и ускорения отклика интерфейса. Для повышения производительности клиентской части также применяются техники кэширования данных, что позволяет сократить количество запросов к серверу при повторных обращениях к одним и тем же данным. Это особенно важно для страниц с часто запрашиваемой информацией, таких как каталог

жилья или профиль пользователя. Кроме того, в клиентской части реализована обработка состояний загрузки, которая отображает индикаторы прогресса или сообщения об ошибках, улучшая пользовательский опыт. Все компоненты интерфейса разработаны с учетом принципов модульности, что упрощает их повторное использование и тестирование. Это позволяет динамически обновлять данные на страницах. Благодаря этому интерфейс остается отзывчивым, а взаимодействие пользователя с приложением – интуитивно понятным.

На рисунке 3.2 представлена структура проекта.

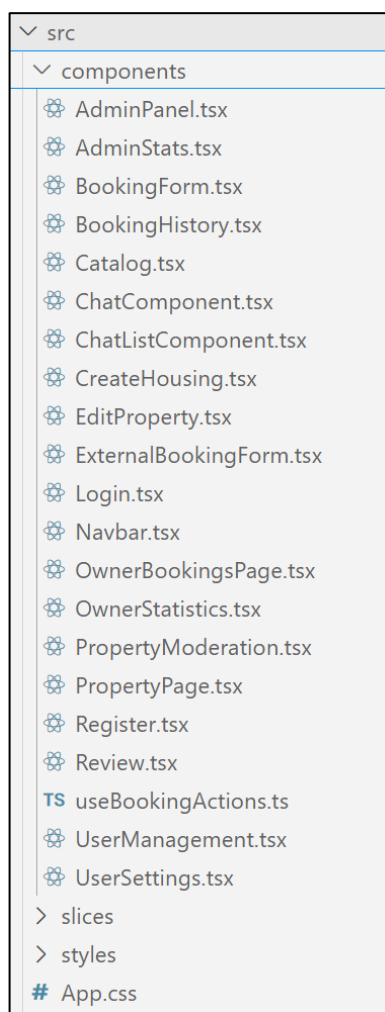


Рисунок 3.2 – Структура проекта

Директория components содержит отдельные файлы компонентов приложения, организованных по функциональному назначению. Это основные части интерфейса. Папка slices содержит userSlice.ts, App.tsx – главный компонент, в котором определены маршруты приложения и подключение основных компонентов, index.tsx – точка входа в клиентское приложение, store.ts – содержит настройку Redux-хранилища для глобального управления состоянием, package.json – файл конфигурации npm, где указаны зависимости, скрипты и метаданные о проекте, tsconfig.json – конфигурационный файл TypeScript, задающий правила компиляции кода.

В листинге 3.10 будет представлен слайс, который управляет состоянием.

```

import { createSlice, PayloadAction } from '@reduxjs/toolkit';
const initialState: UserState = {
  role: localStorage.getItem('role') || null,
  userName: localStorage.getItem('userName') || null,
  userId: localStorage.getItem('userId') || null,
};
const userSlice = createSlice({
  name: 'user',
  initialState,
  reducers: {
    setUserRole(state, action: PayloadAction<string>) {
      state.role = action.payload;
      localStorage.setItem('role', action.payload); // Сохра-
няем роль в localStorage
    },
    setUserName(state, action: PayloadAction<string>) {
      state.userName = action.payload;
      localStorage.setItem('userName', action.payload);
    },
    setUserId(state, action: PayloadAction<string>) {
      state.userId = action.payload;
      localStorage.setItem('userId', action.payload);
    },
    logout(state) {
      state.role = null;
      state.userName = null;
      state.userId = null;
      localStorage.removeItem('role');
      localStorage.removeItem('userName');
      localStorage.removeItem('userId');
    },
  },
});

```

Листинг 3.10 – Слайс для хранения состояния

Константа `userSlice` представляет собой модуль состояния (часть хранилища) для управления данными о пользователе в Redux. В этом модуле хранятся три основных свойства: роль пользователя (`role`), имя пользователя (`userName`) и идентификатор пользователя (`userId`). Эти данные также сохраняются в `localStorage`, чтобы оставаться доступными даже при перезагрузке страницы. Инициализация состояния происходит с извлечением этих значений из `localStorage`, если они там есть.

Слайс включает несколько функций обработки данных (редьюсеров): `setUserRole` обновляет роль пользователя в состоянии и сохраняет ее в `localStorage`, `setUserName` обновляет имя пользователя и также сохраняет его в `localStorage`, `setUserId` обновляет идентификатор пользователя и сохраняет его в `localStorage`, `logout` сбрасывает данные о пользователе в состоянии и удаляет информацию из `localStorage`, что позволяет реализовать выход из системы. Эти функции обновляют состояние приложения, связанное с пользователем

Редьюсер – это функция, которая отвечает за изменение состояния в приложении на основе действий, происходящих в системе. В данном случае, каждая из этих функций обновляет определенное свойство состояния пользователя и сохраняет его в `localStorage` для доступности данных при перезагрузке страницы.

Состояние хранится в `Redux`, и данные о пользователе могут быть легко обновлены с помощью этих редьюсеров.

Объект хранилища `store` будет представлен в листинге 3.11.

```
import { configureStore } from '@reduxjs/toolkit';
import userReducer from './slices/userSlice';
export const store = configureStore({
  reducer: {user: userReducer,},});
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Листинг 3.11 – Код хранилища

Переменная `store` – это конфигурация `Redux`-хранилища, которое управляет глобальным состоянием приложения. Хранилище настраивается с использованием `configureStore` из библиотеки `@reduxjs/toolkit`. В данном случае, хранилище включает редьюсер пользователя (`userReducer`), который управляет состоянием пользователя.

Для удобства работы с состоянием и диспетчером в коде также определены типы: `RootState` – тип, описывающий все состояние приложения, который позволяет использовать типизацию при доступе к состоянию, `AppDispatch` – тип диспетчера, который используется для отправки экшенов в `Redux`.

Таким образом, `store` централизует управление состоянием приложения и позволяет эффективно работать с данными о пользователе.

3.3 Выводы по разделу

В данном разделе рассмотрены структура и технологии как серверной, так и клиентской частей приложения. Серверная часть основана на `Node.js`, который реализует регистрацию, авторизацию, управлением жильем, бронирование жилья и оставление отзывов. Клиентская часть построена на `React` с `React-Redux` для управления состоянием, а маршрутизация и асинхронная загрузка страниц реализованы с использованием `react-router-dom`.

4 Тестирование веб-приложения

Тестирование приложения играет ключевую роль, не уступая по значимости процессу его разработки, так как позволяет выявить и устранить недочеты в работе системы. В данном разделе будут рассмотрены методы проведения тестирования серверной и клиентской частей дипломного проекта.

Необходимо проверить правильность выполнения основных операций, доступных пользователям, а также убедиться, что система корректно обрабатывает некорректные действия, предотвращая их и уведомляя об ошибках. Для минимизации вероятности возникновения сбоев каждый компонент системы будет протестирован индивидуально.

4.1 Тестирование авторизации и регистрации

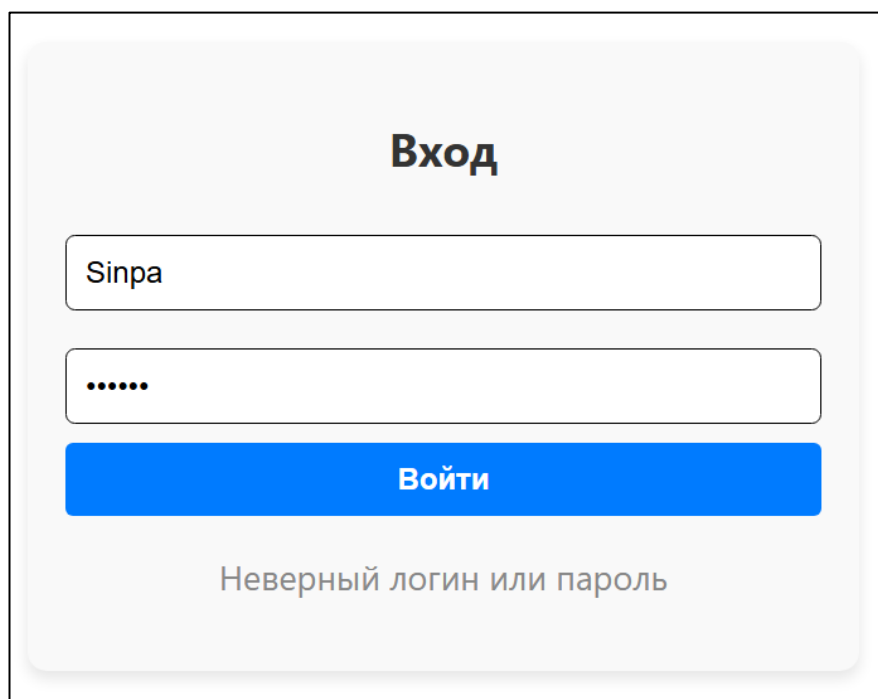
При тестировании данного приложения, были применены сценарии, которые могли бы привести к ошибке. В этой главе рассмотрим некоторые такие сценарии и посмотрим на их обработку.

В момент авторизации, возможна такая ситуация, в которой пользователь ничего не ввел. Обработка данного сценария приведена на рисунке 4.1.

Рисунок 4.1 – Обработка пустых полей при авторизации

При вводе некорректных данных, то есть данные, которые не хранятся в базе данных, возникает ошибка, представленная на рисунке 4.2.

					ДП 04.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.	Синяк П.С.				4 Тестирование веб-приложения	Лит.	Лист	Листов
Пров.	Комкова А.В.					У	1	8
						БГТУ 1-40 05 01, 2025		
Н. контр.	Алешаускас В.А.							
Утв.	Блинова Е.А.							



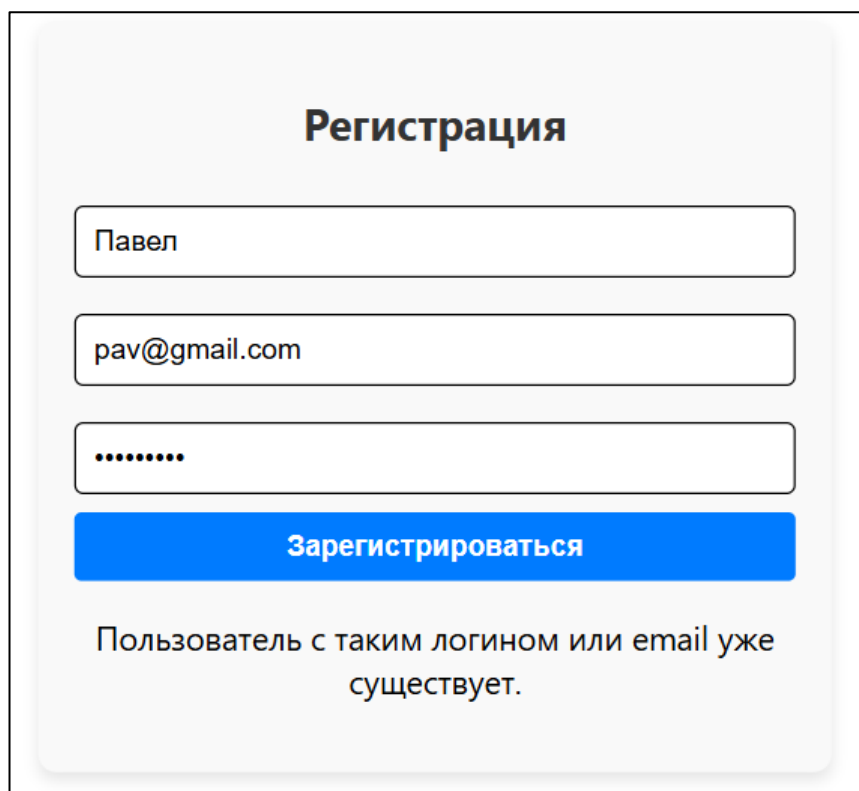
Вход

Войти
Неверный логин или пароль

Рисунок 4.2 – Обработка некорректных данных

При регистрации может быть попытка создать аккаунт с логином, который уже занят, поэтому было принято решение проверить это.

Предусловием будет то, что уже создан аккаунт с логином Павел, но будет попытка создать еще один аккаунт с таким логином. Результат данного теста представлен на рисунке 4.3.



Регистрация

Зарегистрироваться
Пользователь с таким логином или email уже существует.

Рисунок 4.3 – Обработка ввода уже зарегистрированного логина

Дальше нужно попробовать ввести некорректные данные. Например, некорректную почту. Результат данного тестирования представлен на рисунке 4.4.

The screenshot shows a registration form with the title "Регистрация". It contains two input fields: the first contains "Sinpa" and the second contains "sinpagmail.com". Below the second field, a red error message is displayed: "Адрес электронной почты должен содержать символ '@'. В адресе 'sinpagmail.com' отсутствует символ '@'." At the bottom of the form is a blue button labeled "Зарегистрироваться".

Рисунок 4.4 – Результат с невалидной электронной почтой

При попытке ввести пароль, который содержит менее 6 символов, будет отображена соответствующая ошибка системы. Результат данного тестирования представлен на рисунке 4.5.

The screenshot shows a registration form with the title "Регистрация". It contains three input fields: the first contains "Sinpa", the second contains "sinpa@gmail.com", and the third contains two dots "••". Below the third field, a red error message is displayed: "Пароль должен содержать не менее 6 символов". At the bottom of the form is a blue button labeled "Зарегистрироваться".

Рисунок 4.5 – Результат с невалидного пароля

Результаты тестирования авторизации и регистрации свидетельствуют о том, что все функции работают корректно.

4.2 Тестирование добавление жилья

Тестирование добавления жилья является важным этапом для проверки корректной работы функционала. Попытка добавить жилье с пустыми всеми полями. В этом случае система должна отклонить запрос и отобразить соответствующее сообщение об ошибке.

На рисунке 4.6 представлен результат тестирования.

The screenshot shows a web form titled "Создать новое жилье" (Create new accommodation). The form contains several input fields, all of which are empty. A red error message "Заполните это поле." (Fill in this field.) is displayed above the "Описание" (Description) field. The form fields are as follows:

- Название:** Input field with placeholder "Название жилья".
- Описание:** Textarea with placeholder "Описание жилья". A red error message "Заполните это поле." is shown above it.
- Местоположение:** Input field with placeholder "Город, адрес".
- Цена за ночь:** Input field with placeholder "Цена в рублях".
- Категория:** Dropdown menu with placeholder "Выберите категорию".
- Критерии:** Input field with placeholder "Поиск по критериям".
- Изображения жилья:** A list of checkboxes for features: "Количество комнат: 1", "Количество комнат: 2", "Количество комнат: 3", "Количество комнат: 3+", "Наличие лифта", and "Wi-Fi".
- Изображения жилья:** A button "Выбрать файлы" and the text "Файл не выбран".
- Документы (PDF, DOC, изображения) подтверждающие жилье:** A button "Выбрать файлы" and the text "Файл не выбран".
- Создать:** A blue button at the bottom of the form.

Рисунок 4.6 – Добавление жилья с пустыми полями

При создании жилья необходимо заполнить обязательные поля. Если они отсутствуют, создание жилья будет невозможно. Помимо этого, при создании жилья обязательно нужно выбрать изображения и документы жилья как показано на рисунке 4.7.

Добавить жилье

Фильтр по категории:
 Все

Критерии:
 Поиск по критериям

☐ Количество комнат: 1
☐ Количество комнат: 2
☐ Количество комнат: 3
☐ Количество комнат: 3+
☐ Наличие лифта
☐ Wi-Fi
☐ Кондиционер

Сортировка:
 По цене

Сбросить фильтры



Просторная квартира у метро

Светлая квартира с двумя спальнями рядом с метро Площадь Якуба Коласа. Полностью оборудована

Местоположение: Минск, ул. Веры Хоружей, 15

Цена за ночь: 100 руб.

Категория: Квартира

Критерии: Wi-Fi, Парковка (бесплатная), Количество комнат: 1

Владелец: Пользователь 111 (111@gmail.com)

Удалить жилье

Рисунок 4.9 – Поиск с определенным именем

Следующим этапом тестирования была проверка фильтрации по категории жилья. На рисунке 4.10 представлен результат данного тестирования.

Добавить жилье

Фильтр по категории:
 Квартира

Критерии:
 Поиск по критериям

☐ Количество комнат: 1
☐ Количество комнат: 2
☐ Количество комнат: 3
☐ Количество комнат: 3+
☐ Наличие лифта
☐ Wi-Fi
☐ Кондиционер

Сортировка:
 По цене

Сбросить фильтры



Просторная квартира у метро

Светлая квартира с двумя спальнями рядом с метро Площадь Якуба Коласа. Полностью оборудована

Местоположение: Минск, ул. Веры Хоружей, 15

Цена за ночь: 100 руб.

Категория: Квартира

Критерии: Wi-Fi, Парковка (бесплатная), Количество комнат: 1

Владелец: Пользователь 111 (111@gmail.com)

Удалить жилье



Уютные апартаменты в центре

Современные апартаменты с панорамными окнами в сердце Минска. Идеально для туристов и деловых поездок

Местоположение: Минск, ул. Немига, 10

Цена за ночь: 150 руб.

Категория: Квартира

Критерии: Наличие лифта, Wi-Fi, Кондиционер

Владелец: Пользователь 111 (111@gmail.com)

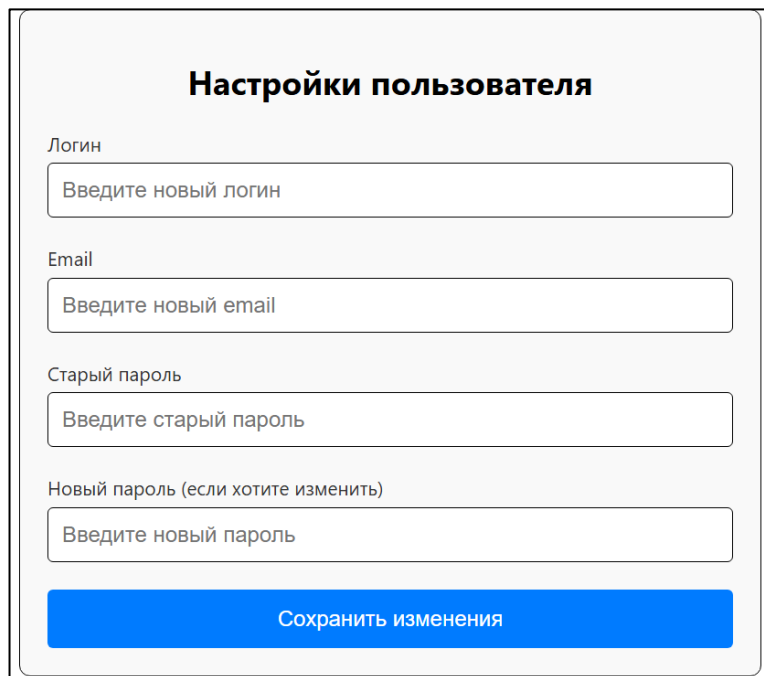
Удалить жилье

Рисунок 4.10 – Фильтрация по категории

Тесты по поиску жилья были проведены успешно, подтверждая корректную работу фильтров и отображение результатов.

4.4 Тестирование изменения настроек пользователя

Для тестирования изменения настроек пользователя был рассмотрен случай, когда логин, который меняется уже занят. На рисунке 4.11 представлен результат данного тестирования.

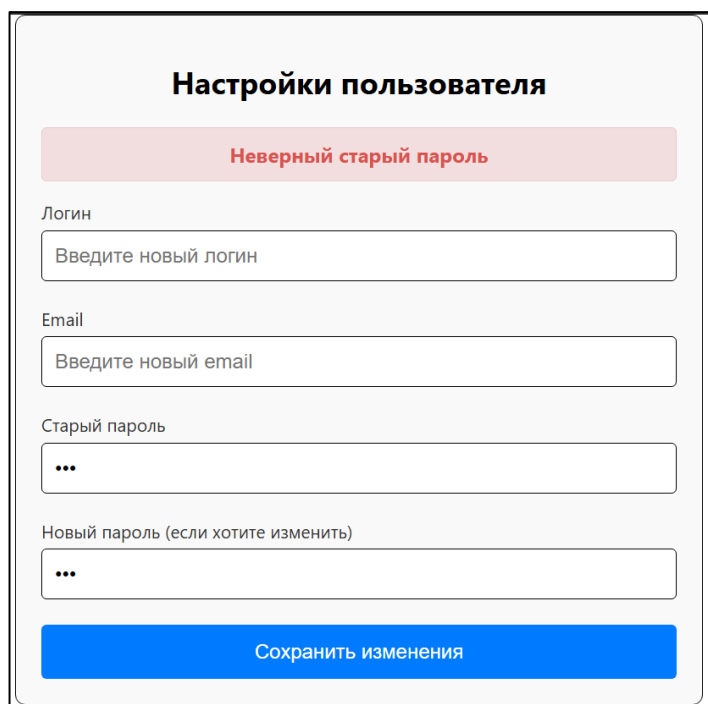


The screenshot shows a web form titled "Настройки пользователя" (User Settings). It contains four input fields: "Логин" (Login) with placeholder "Введите новый логин", "Email" with placeholder "Введите новый email", "Старый пароль" (Old password) with placeholder "Введите старый пароль", and "Новый пароль (если хотите изменить)" (New password (if you want to change)) with placeholder "Введите новый пароль". At the bottom is a blue button labeled "Сохранить изменения" (Save changes).

Рисунок 4.11 – Проверка ввода существующего логина

Далее была проведена проверка на возможность изменения пароля, введя некорректный старый пароль.

На рисунке 4.12 представлен результат данного тестирования.



The screenshot shows the same "Настройки пользователя" form. At the top, below the title, there is a red error message: "Неверный старый пароль" (Incorrect old password). The input fields for "Логин", "Email", "Старый пароль", and "Новый пароль" are present, but the "Старый пароль" and "Новый пароль" fields now contain three dots "...". The "Сохранить изменения" button remains at the bottom.

Рисунок 4.12 – Результат некорректного ввода старого пароля

Тест прошел успешно, так как для смены пароля необходимо ввести корректный старый пароль, который подтвердит, что вы являетесь настоящим владельцем данного аккаунта.

4.5 Тестирование бронирования жилья

Для бронирования жилья был рассмотрен случай, когда пользователь хочет забронировать жилье на занятые даты. На рисунке 4.13 представлен результат данного тестирования.

Рисунок 4.13 – Проверка возможности выбрать занятые даты

Тест пройден как надо, и даты, на которые уже существует бронирование выбрать нет возможности, что свидетельствует об успешном прохождении теста.

4.6 Выводы по разделу

Таким образом, были проведены тестирования нового функционала, включая добавление жилья, поиск жилья, а также изменения настроек пользователя. Все тесты прошли успешно, функционал работает корректно. Система правильно обрабатывает обязательные поля, фильтрацию данных и обработку новых данных. В случае ввода некорректных данных система корректно генерирует ошибки и предотвращает сохранение неверных данных. Проведенные тесты подтвердили правильность работы всех функций, что свидетельствует о высокой надежности и безопасности приложения.

5 Руководство пользователя (руководство по эксплуатации)

5.1 Руководство для гостя

При открытии веб-приложения, первой страницей является страница каталога. Если у гостя еще нет аккаунта, он может перейти на страницу регистрации, где необходимо ввести свой логин, почту и пароль. Страница регистрации, на которой указаны данные поля, представлена на рисунке 5.1.

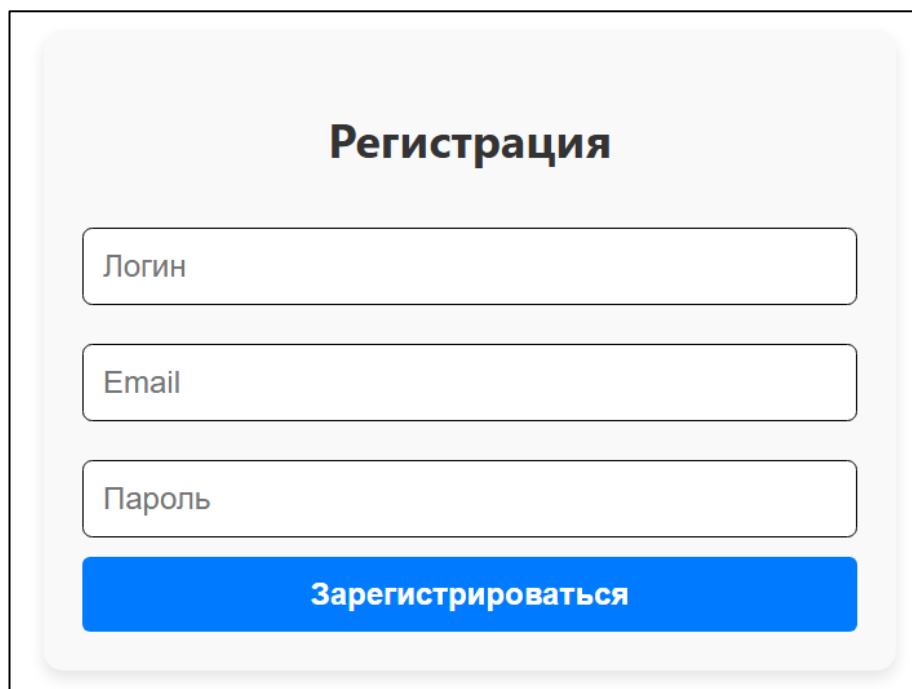


Рисунок 5.1 – Страница регистрации

После регистрации гостю присваивается роль обычного пользователя, то есть человека, который собирается арендовать жилье через данное приложение.

5.2 Руководство для пользователя

Пользователь также, как и гость, может просматривать каталог жилья, искать по названию, сортировать или фильтровать жилье по своему усмотрению. Помимо этого, в шапке сайта можно будет увидеть кнопку «Каталог» для перехода в каталог, имя пользователя, его роль в данный момент, кнопка «История бронирований», кнопка «Пройти верификацию», кнопка «Настройки пользователя», кнопка «Мои чаты» и кнопка «Выйти». На рисунке 5.2 и в приложении Е представлена страница каталога пользователя.

					ДП 05.00.ПЗ		
		ФИО	Подпись	Дата	5 Руководство пользователя (руководство по эксплуатации)		
Разраб.	Синяк П.С.						
Пров.	Комкова А.В.						
Н. контр.	Алешаускас В.А.						
Утв.	Блинова Е.А.						
					Лит.	Лист	Листов
					У	1	12
					БГТУ 1-40 05 01, 2025		

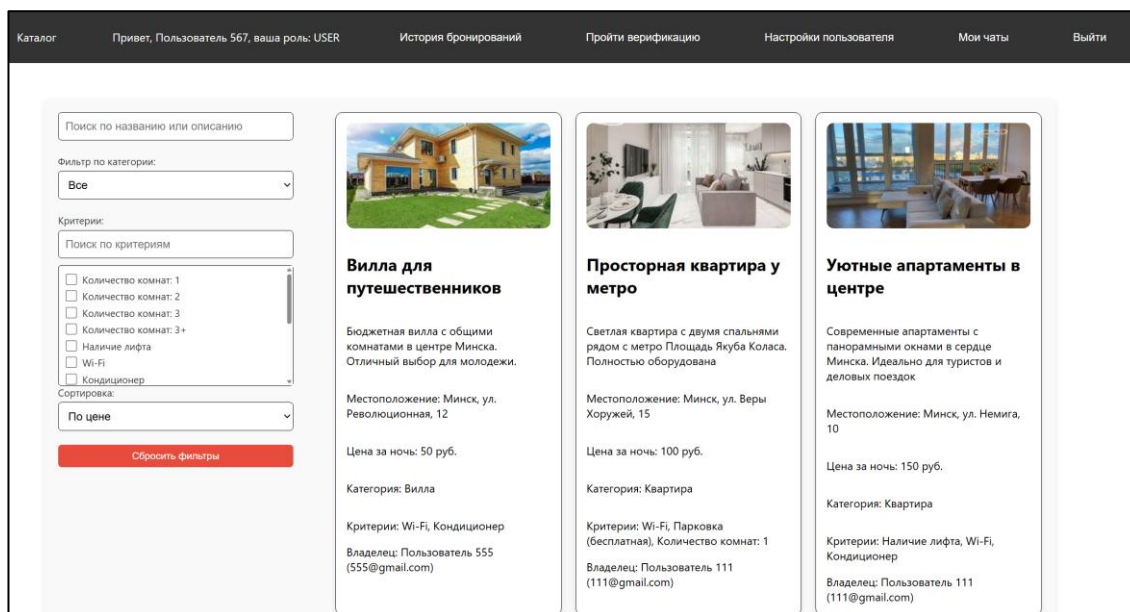


Рисунок 5.2 – Каталог жилья

Кликнув по карточке жилья, пользователь сможет подробнее ознакомиться с деталями, а после этого подать запрос на бронирование. Если бронирование будет подтверждено, у пользователя появится возможность оставить отзыв под этим жильем, также он может видеть историю бронирований данного жилья. На рисунке 5.3 представлена данная страница, где показана история бронирований, календари, в которых пользователь выбирает даты бронирования, а также раздел с отзывами.

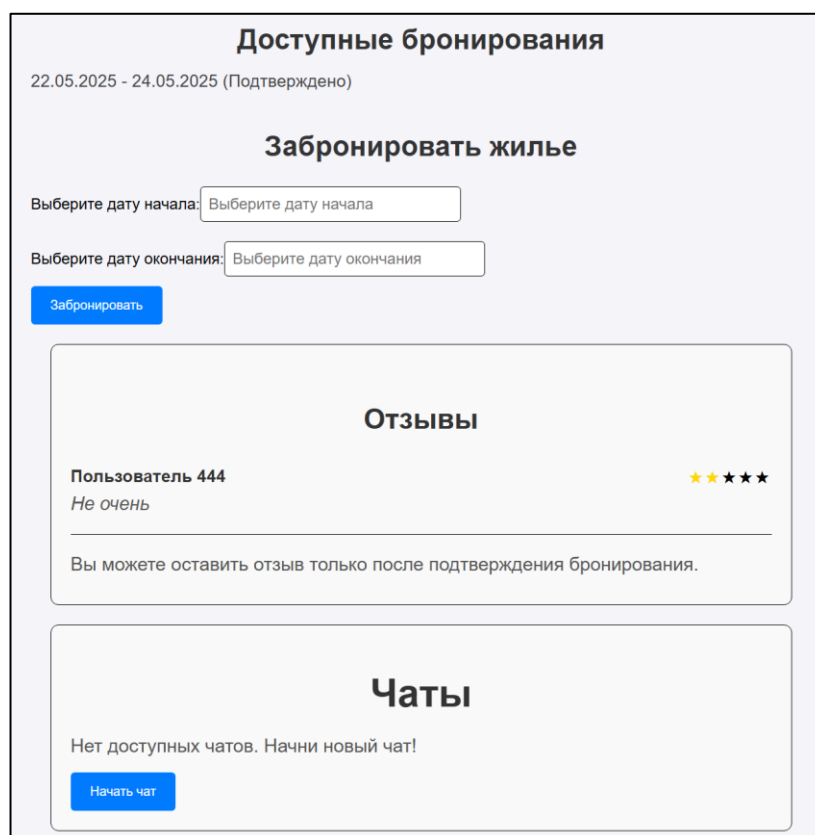


Рисунок 5.3 – Страница жилья

Как уже было сказано до этого в шапке сайта есть кнопка «История бронирований», на которой будет отображаться список всех бронирований данного пользователя. Их можно отсортировать по дате и алфавиту, а также отфильтровать по статусу бронирования. На рисунке 5.4 представлена эта страница.

История бронирований

По дате ▾ Все ▾

Коттедж уютный
18.05.2025 - 19.05.2025 (В ожидании)
Цена за ночь: 250 руб.

Вилла для путешественников
21.05.2025 - 25.05.2025 (Подтверждено)
Цена за ночь: 50 руб.

Вилла для путешественников
31.05.2025 - 01.06.2025 (Подтверждено)
Цена за ночь: 50 руб.

Рисунок 5.4 – Страница истории бронирования

Помимо этого, есть кнопка «Настройки пользователя», где можно изменить логин, почту или пароль. Система проверит есть ли данный логин или почта в базе данных и оповестит пользователя, что данные изменены, либо, при обнаружении совпадений, оповестит об ошибке. В случае желания смены пароля, пользователю необходимо будет ввести старый пароль. На рисунке 5.5 представлена данная страница с возможностью смены логина, почты и пароля.

Настройки пользователя

Логин
Введите новый логин

Email
Введите новый email

Старый пароль
Введите старый пароль

Новый пароль (если хотите изменить)
Введите новый пароль

Сохранить изменения

Рисунок 5.5 – Страница настроек пользователя

Помимо всего прочего есть кнопка «Мои чаты», кликнув на которую, пользователь сможет увидеть список чатов, в которых участвует этот пользователь. Данная страница представлена на рисунке 5.6.

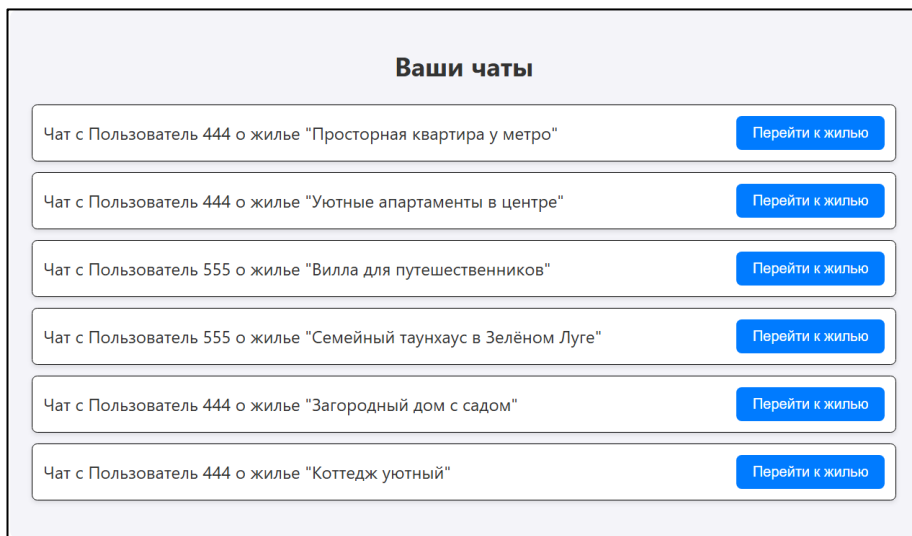


Рисунок 5.6 – Страница с чатами пользователя

Если нажать на кнопку «Перейти к жилью», то откроется страница с деталями о жилье. Внизу будет отображаться активный чат с владельцем этого жилья. Если его нет, то будет кнопка «Создать чат». Данный функционал представлен на рисунке 5.7.

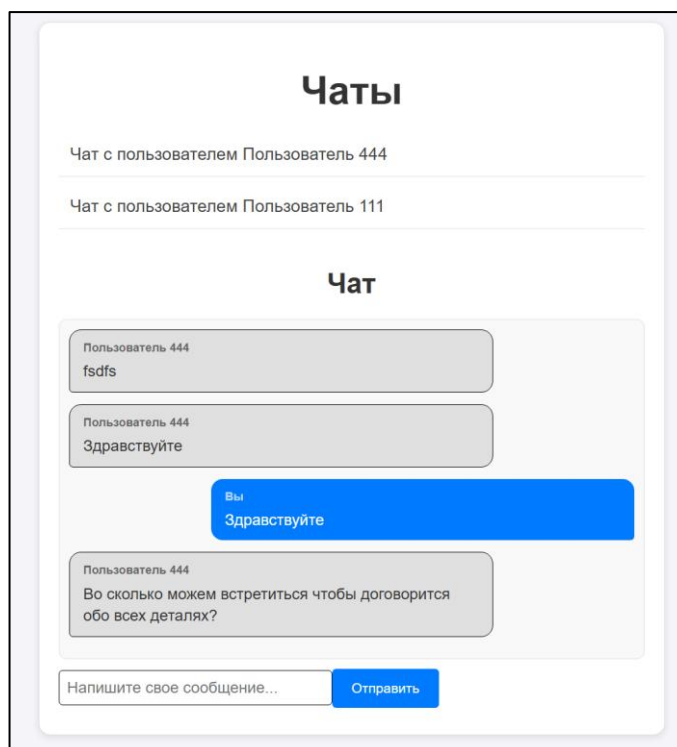


Рисунок 5.7 – Компонент чата

Таким образом, в данном разделе был рассмотрен весь функционал пользователя, доступный в этом приложении.

5.3 Руководство для владельца

Для того чтобы стать владельцем жилья, пользователю необходимо пройти верификацию. Для этого в шапке нужно нажать на кнопку «Пройти верификацию», после чего всплывет окошко, в котором необходимо будет выбрать документ для подтверждения личности, как показано на рисунке 5.8. После чего останется только дождаться, пока заявку одобрит администратор и затем нажать на кнопку «Стать владельцем» в шапке приложения.

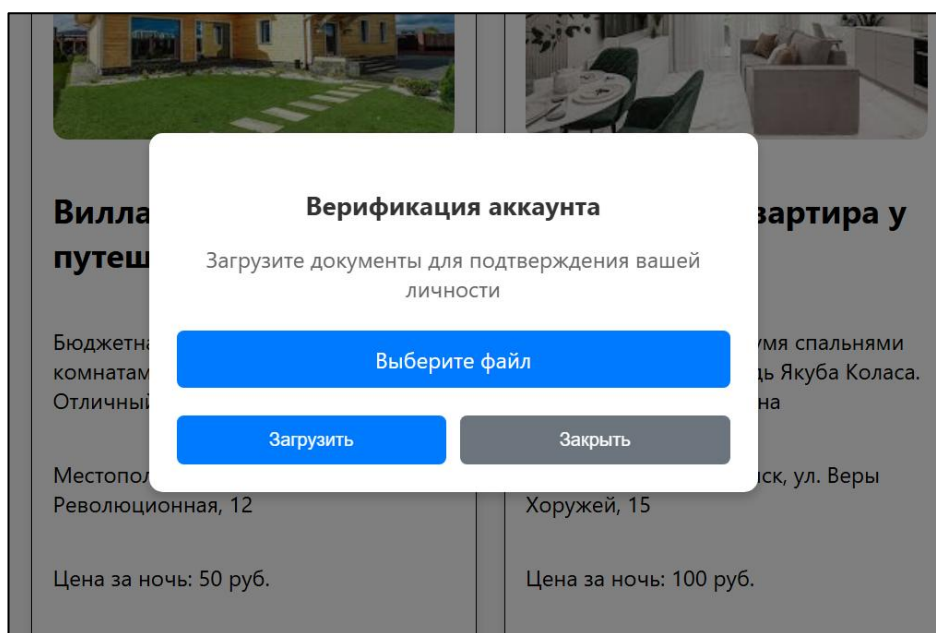


Рисунок 5.8 – Окно с верификацией

Став владельцем, каталог жилья изменится, отображая только жилье, которое находится во владении данного пользователя. Помимо этого, появится кнопка «Добавить жилье», а также кнопка «Удалить жилье». На рисунке 5.9 представлен каталог с новой кнопкой.

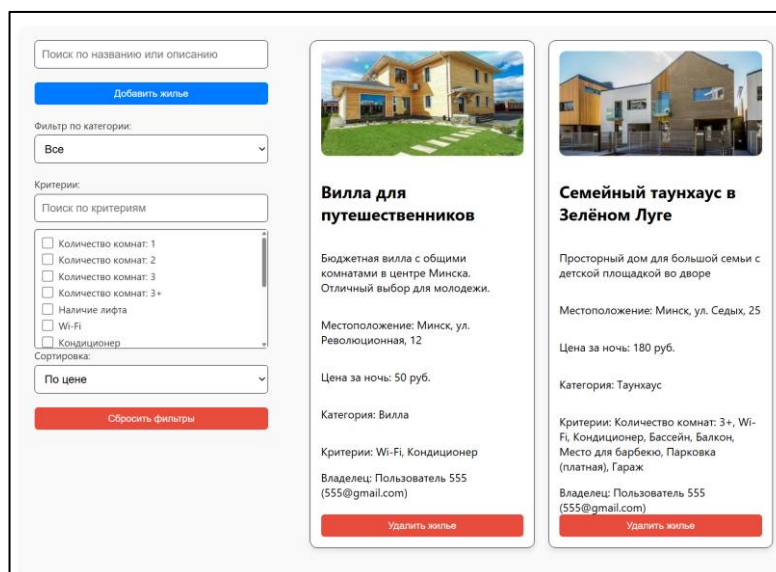


Рисунок 5.9 – Кнопка «Добавить жилье»

Нажав на кнопку «Добавить жилье», пользователь откроет страницу, на которой потребуется ввести название жилья, описание, местоположение, цену за ночь, выбрать категорию жилья, при необходимости выбрать определенные критерии, выбрать изображения жилья, а также документ, который бы подтверждал право собственности на жилье, после чего нажать на кнопку «Создать». На рисунке 5.10 представлена страница создания жилья. После чего жилье создастся, но только в каталоге самого владельца, а когда жилье одобрит администратор, оно появится для всех в каталоге.

Создать новое жилье

Название:

Описание:

Местоположение:

Цена за ночь:

Категория:

Критерии:

☐ Количество комнат: 1
☐ Количество комнат: 2
☐ Количество комнат: 3
☐ Количество комнат: 3+
☐ Наличие лифта
☐ Wi-Fi

Изображения жилья:

Документы (PDF, DOC, изображения) подтверждающие жилье:

Рисунок 5.10 – Создание жилья

Помимо этого, страница жилья также претерпела некоторые изменения у владельца. Появилась возможность удаления данного жилья, редактирования, есть возможность принять или отменить бронирование, а также создать собственное бронирование для пользователя, например, если с владельцем напрямую свяжутся и попросят об этом, как показано на рисунке 5.11.

Рисунок 5.11 – Страница информации о жилье для владельца

Страница редактирования будет содержать старые данные о жилье, а именно: название, описание, местоположение, цену за ночь, категорию, критерии, а также изображения. Данная страница представлена на рисунке 5.12.

Рисунок 5.12 – Страница редактирования жилья

Итак, будучи владельцем, когда приходит очередной запрос на бронирование, в шапке сайта будет появляться кнопка, оповещающая, сколько активных запросов в данный момент. Нажав на данную кнопку, владелец окажется на странице, в которой будут активные запросы на бронирование у каждого жилья с возможностью перейти в конкретное жилье, как показано на рисунке 5.13.

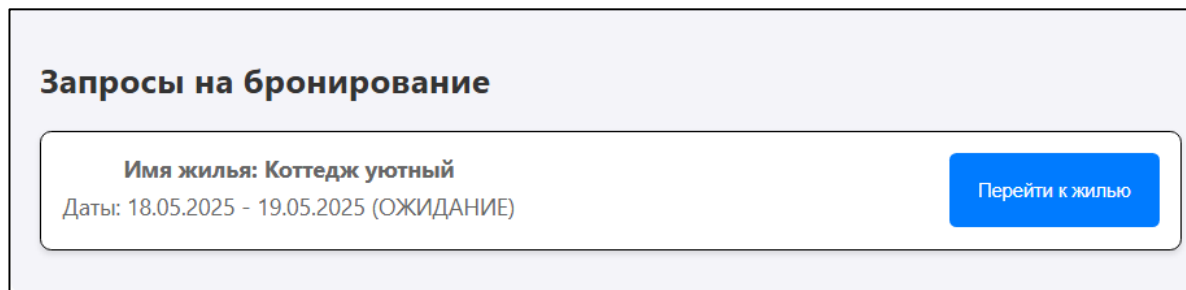


Рисунок 5.13 – Страница с активными запросами на бронирование

И как уже понятно, владелец может нажать на кнопку «Стать пользователем», чтобы снова поменять свою роль, если появилась необходимость арендовать жилье. Стоит отметить, что, будучи пользователем, в каталоге он не сможет увидеть свое жилье.

Также в шапке есть кнопка «Статистика владельца», нажав на которую, откроется страница, которая представлена на рисунке 5.14.

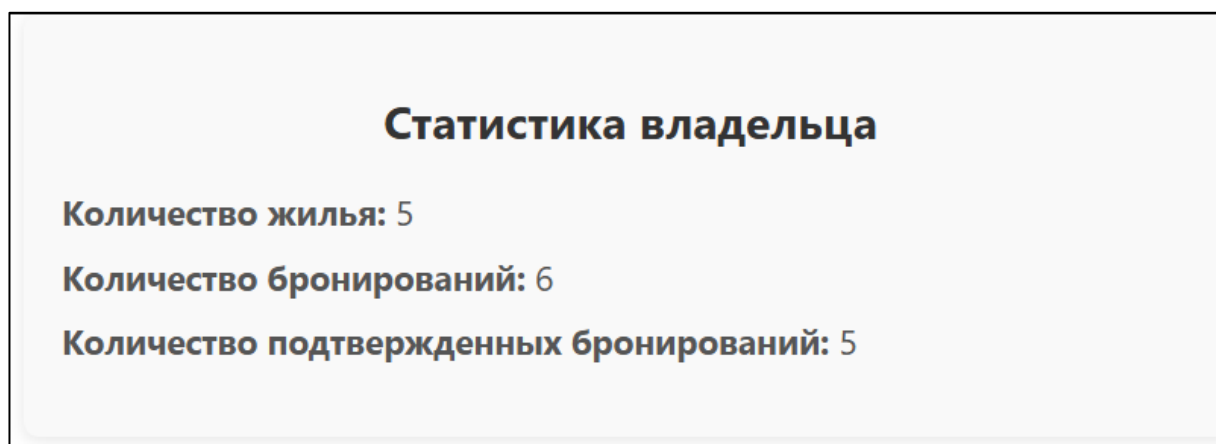


Рисунок 5.14 – Страница со статистикой владельца

На данной странице представлено количество жилья данного владельца, количество бронирований и количество подтвержденных бронирований.

Стоит отметить, что кнопки «Мои чаты» и настройки пользователя функционируют так же, как и для роли пользователя.

5.4 Руководство для администратора

После авторизации администратор окажется на странице каталога жилья, где он, так же, как и владелец, будет иметь возможность удалять жилье. Помимо этого, в шапке сайта появится новая кнопка «Панель администратора», как показано на рисунке 5.15.

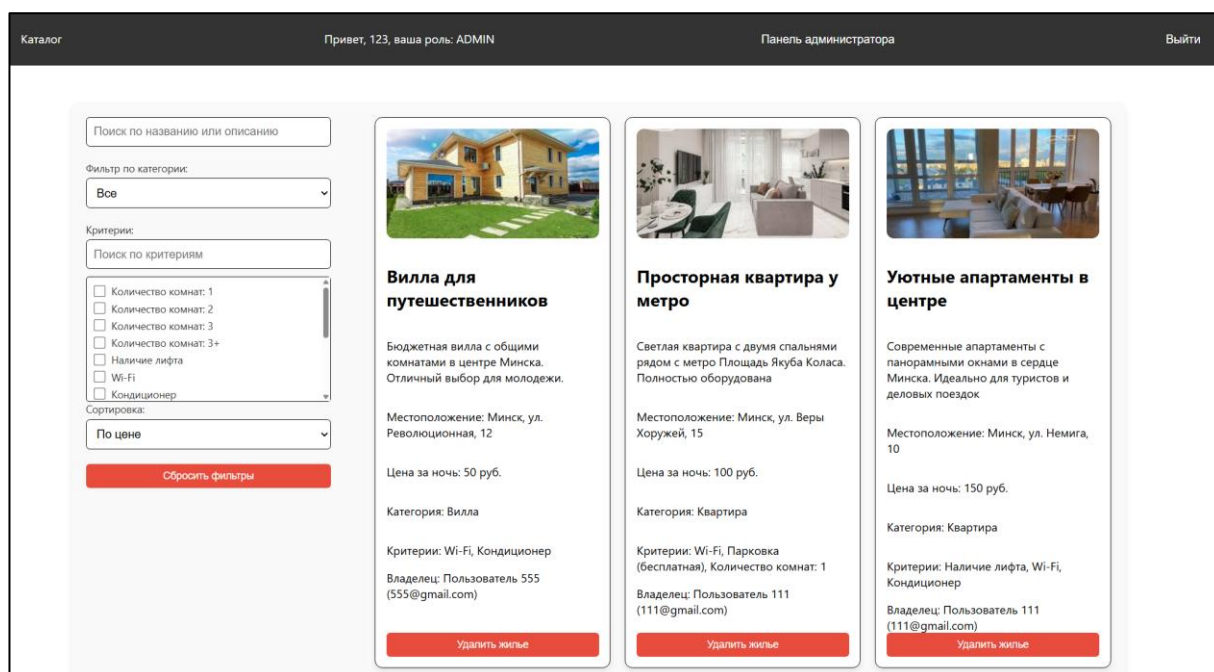


Рисунок 5.15 – Каталог для администратора

Перейдя в панель администратора, можно будет добавлять новые категории и критерии жилья, а также удалять их, как показано на рисунке 5.16.

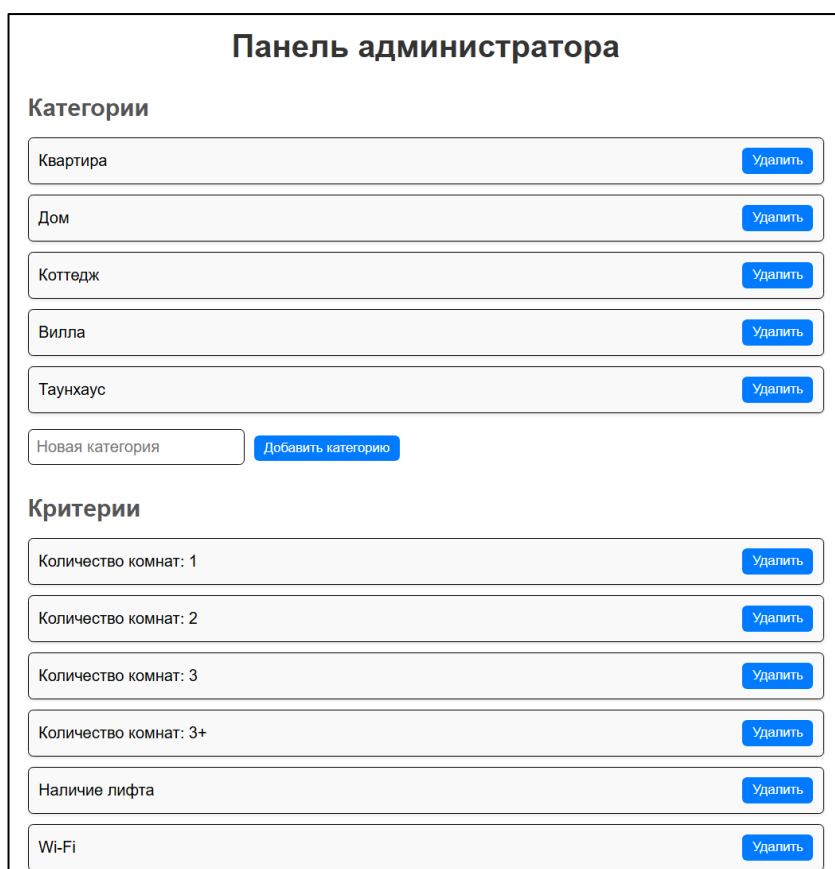


Рисунок 5.16 – Панель администратора

Дальше будет отображаться список модерируемого жилья. То есть тут будет список жилья и прикрепленный документ, который администратор может

скачать и ознакомиться, после чего вынести решение касательно одобрения или отклонения жилья. Данный процесс представлен на рисунке 5.17.

Модерация жилья

Ожидающие модерации

Уютные апартаменты в центре

Современные апартаменты с панорамными окнами в сердце Минска. Идеально для туристов и деловых поездок

Документы:

[housingDocuments-1747583187307-96434645.docx](#)

Одобрить

Отклонить

Просторная квартира у метро

Светлая квартира с двумя спальнями рядом с метро Площадь Якуба Коласа. Полностью оборудована

Документы:

[housingDocuments-1747583352373-93694761.pdf](#)

Одобрить

Отклонить

Рисунок 5.17 – Модерация жилья

Также тут будут отображаться все пользователи и будет возможность заблокировать и разблокировать пользователя, а также скачать документ, удостоверяющий личность пользователя, если он хочет пройти верификацию, и, соответственно, будет кнопка «Подтвердить» для таких пользователей. Если пользователь заблокирован, после выхода из аккаунта заново войти он не сможет, пока не будет разблокирован. Данный компонент представлен на рисунке 5.18.

Список пользователей						
Имя	Почта	Роль	Статус	Документ	Действия	
5555	55@gmail.com	USER	Подтвержден	Скачать документ	Заблокировать	
987	987@gmail.com	USER	Не подтвержден		Подтвердить	Заблокировать
777	777@gmail.com	USER	Подтвержден	Скачать документ	Заблокировать	
001	001@gmail.com	OWNER	Подтвержден	Скачать документ	Заблокировать	
555	555@gmail.com	USER	Подтвержден	Скачать документ	Заблокировать	
321	321@gmail.com	USER	Подтвержден		Заблокировать	
1234	123@gmail.com	OWNER	Подтвержден		Заблокировать	
12345	12345@gmail.com	USER	Не подтвержден		Подтвердить	Заблокировать
444	444@gmail.com	OWNER	Подтвержден		Заблокировать	
000	000@gmail.com	USER	Подтвержден		Заблокировать	
456	456@gmail.com	OWNER	Подтвержден	Скачать документ	Заблокировать	
111	111@gmail.com	OWNER	Подтвержден		Заблокировать	

Рисунок 5.18 – Управление пользователями

Помимо этого можно посмотреть статистику, на которой представлено количество пользователей, количество жилья, а также статистика по каждой категории жилья, как показано на рисунке 5.19.

Статистика для администратора

Количество пользователей: 14

Количество жилья: 6

Статистика по категориям:

Квартира: 2 объектов

Дом: 1 объектов

Коттедж: 1 объектов

Вилла: 1 объектов

Таунхаус: 1 объектов

Рисунок 5.19 – Статистика для администратора

Таким образом, выглядит руководство пользователя для администратора.

5.5 Установка приложения

Для запуска приложения необходимо выполнить определенные шаги.

Запустить серверную часть приложения, которая соединяет базу данных и React-приложение. Для этого необходимо запустить скрипт, который настроит соединение с базой данных и запустит сервер. Перед запуском сервера нужно настроить файл `.env`, содержащий конфигурационные параметры для подключения к базе данных и другие настройки.

Файл `.env` – это текстовый файл, используемый для хранения переменных окружения, таких как строка подключения к базе данных, порты сервера и другие конфиденциальные данные. Он позволяет отделить настройки от кода приложения, что упрощает управление конфигурацией и повышает безопасность. Пример содержимого файла `.env` представлен в листинге 5.1.

```

DATABASE_URL=postgresql://admin:secret@localhost:5432/my-
app?schema=public
SERVER_PORT=5000

```

Листинг 5.1 –Содержимое файла `.env`

Переменная `DATABASE_URL`: это полная строка подключения к базе данных, включающая адрес сервера (например, `localhost`), порт (например, 5432 для PostgreSQL), имя пользователя (`admin`), пароль (`secret`), имя базы

данных (myapp) и, при необходимости, схему (например, public). Переменная `SERVER_PORT`: это порт, на котором будет работать сервер приложения.

Эти переменные загружаются в приложение с помощью библиотек, таких как `dotenv` (в `Node.js`), и используются в скрипте для настройки соединения с базой данных. `Prisma`, как ORM, напрямую использует `DATABASE_URL` для подключения. Файл `.env` не должен добавляться в систему контроля версий (например, в `Git`), поэтому его обычно включают в файл `.gitignore`.

Также перед запуском сервера необходимо подготовить базу данных, а именно создать ее в своей СУБД. `Prisma` использует миграции для управления структурой базы данных. Миграции позволяют определить схему базы данных в файле `schema.prisma` и синхронизировать ее с реальной базой данных. Для этого необходимо выполнить команду `prisma migrate dev`, таким образом в созданной ранее базе данных будут созданы необходимые таблицы.

Теперь остается только запустить сервер командой `npm run start`. Затем запустить `React`-приложение, которое будет обрабатывать пользовательские запросы и взаимодействовать с сервером. Для этого необходимо выполнить команду `npm start` для сборки и запуска `React`-приложения.

После выполнения этих шагов приложение будет полностью готово к работе, и пользователь сможет начать использовать его функционал.

5.6 Выводы по разделу

В данном разделе были рассмотрены руководства пользователя для всех ролей: пользователя, владельца и администратора. Для каждой роли были подробно описаны доступные функции и действия, которые могут быть выполнены на платформе, начиная с регистрации и профиля, заканчивая управлением жильем, бронированием и оставлением отзывов. С помощью примеров и изображений показано, как правильно использовать приложение и какие действия доступны пользователям в зависимости от их роли. Таким образом, был наглядно продемонстрирован процесс работы с платформой, что делает использование приложения интуитивно понятным и удобным для всех категорий пользователей.

6 Технико-экономическое обоснование проекта

6.1 Общая характеристика разрабатываемого программного средства

В рамках данного дипломного проекта разработано веб-приложение для аренды жилья, предназначенное для упрощения процесса поиска, бронирования и управления арендой недвижимости.

Целью приложения является предоставление удобной платформы для взаимодействия арендаторов и владельцев жилья, обеспечивая функционал для поиска, бронирования, управления объектами недвижимости и коммуникации между обеими сторонами.

Целевая аудитория приложения включает широкий круг пользователей: арендаторов, таких как студенты, ищущие доступное жилье, туристы, нуждающиеся в краткосрочной аренде, семьи, подбирающие долгосрочное жилье, а также владельцев недвижимости, стремящихся эффективно управлять своими объектами и находить надежных арендаторов.

Пользователи имеют возможность бронировать жилье, общаться с владельцами жилья через встроенный чат, просматривать статистику бронирований, оставлять отзывы и редактировать личную информацию. Владельцы жилья могут добавлять, редактировать и удалять объекты жилья, управлять бронированиями, одобрять или отменять запросы, общаться с арендаторами через чат и просматривать статистику.

Разработанное программное средство имеет следующие преимущества перед аналогичными решениями, рассмотренными в главе 1:

- интуитивно понятный интерфейс, специально адаптированный для удобства разных ролей пользователей;
- высокая степень автоматизации процессов бронирования и управления жильем для упрощения взаимодействия;
- встроенный чат для оперативной коммуникации между арендаторами и владельцами с удобным интерфейсом;
- гибкая система фильтрации и поиска жилья;
- обеспечение безопасности данных пользователей и транзакций.

Стратегия монетизации предполагает показ рекламы клиенту перед каждым входом в приложение для сохранения ссылки.

В рамках данного раздела необходимо определить затраты на всех стадиях разработки программного средства. Также необходимо выполнить расчет экономии основных видов ресурсов в связи с использованием разработанного программного средства на основе успеха и монетизации приложений аналогов.

					ДП 06.00.ПЗ		
		ФИО	Подпись	Дата			
Разраб.		Синяк П.С.			6 Технико-экономическое обоснование проекта		
Пров.		Комкова А.В.					
Консульт.		Соболевский А.С.					
Н. контр.		Алешаускас В.А.					
Утв.		Блинова Е.А.					
					Лит.	Лист	Листов
					У	1	9
					БГТУ 1-40 05 01, 2025		

6.2 Исходные данные для проведения расчётов

Источниками исходных данных для данных расчетов выступают действующие законы и нормативно-правовые акты. Исходные данные для расчета приведены в таблице 6.1.

Таблица 6.1 – Исходные данные для расчета

Наименование показателя	Условные обозначения	Норматив
Норматив дополнительной заработной платы, %	$H_{дз}$	8
Ставка отчислений в Фонд социальной защиты населения, %	$H_{фсзн}$	34,0
Ставка отчислений по обязательному страхованию в БРУСП «Белгосстрах», %	$H_{бгс}$	0,6
Норматив прочих прямых затрат, %	$H_{пз}$	21
Норматив накладных расходов, %	$H_{нр}$	41
Норматив расходов на сопровождение и адаптацию, %	$H_{рр}$	10
Ставка НДС, %	$H_{ндс}$	20,0
Налог на прибыль, %	$H_{п}$	20,0

В ходе проведения маркетингового анализа была установлена стоимость разработки программного продукта на основе стоимости некоторых аналогов.

Таблица 6.2 – Исходные данные аналогов

Наименование приложения	Описание приложения
NedvRf	Платформа для поиска недвижимости, включая аренду жилья, ориентированная на российский рынок. Предлагает базовые фильтры по цене и местоположению, с минималистичным интерфейсом и отсутствием функций, таких как автоматизированное бронирование или чат.
TukTuk	Сервис краткосрочной аренды жилья в Москве, ориентированный на прямое взаимодействие между арендаторами и собственниками. Поддерживает базовый поиск по объявлениям и минимальные фильтры.
Kvartirka	Локальная платформа для краткосрочной аренды жилья в российских городах. Сосредоточена на простом поиске по городам и ценам, с минимальным интерфейсом и прямым контактом с арендодателями, без продвинутых функций бронирования или аналитики.

В стоимость разработки входит создание серверной и клиентской частей приложения. Оценка стоимости основана на анализе аналогичных платформ и технологий, использованных для их создания. Основная сложность оценки заключается в конфиденциальности данных о коммерческих проектах, что ограничивает доступ к точным цифрам. Примерная стоимость разработки программного продукта, выбранного в качестве базы сравнения, составит от 10000 белорусских рублей.

6.3 Обоснование цены программного средства

6.3.1 Расчет затрат рабочего времени на разработку программного средства

Для обоснования целесообразности разработки веб-приложения для аренды жилья требуется комплексный расчет производственных затрат. Формирование полнофункциональной системы, включающей функции поиска, бронирования, управления жильем и коммуникации через чат, предусматривает следующие категории расходов:

- основная заработная плата исполнителей;
- дополнительная заработная плата;
- отчисления в Фонд социальной защиты населения;
- отчисления по обязательному страхованию от несчастных случаев;
- затраты на программное обеспечение и хостинг;
- прочие прямые затраты и накладные расходы.

При создании веб-приложения использовались оптимизированные подходы к разработке, ориентированные на локальный рынок и упрощенный функционал, что позволило сократить затраты на разработчиков и инфраструктуру. Затраты рабочего времени на разработку программного средства представлены в таблице 6.3.

Таблица 6.3 – Затраты рабочего времени на разработку ПС

Содержание работ	Исполнитель	Затраты рабочего времени, часов
Анализ конкурентов и рынка	Бизнес-аналитик	12
Определение пользовательских сценариев	Бизнес-аналитик	10
Составление технического задания	Бизнес-аналитик	25
Проектирование архитектуры сервиса	Системный архитектор	20
Разработка API для бронирования и управления жильем	Backend-разработчик	35
Реализация системы ролей и авторизации	Backend-разработчик	25
Разработка модуля чата	Backend-разработчик	20
Настройка серверной инфраструктуры	DevOps-инженер	35
Проектирование UI/UX прототипа	UX/UI-дизайнер	20
Создание финального дизайн-макета	UX/UI-дизайнер	35
Верстка основных страниц	Frontend-разработчик	40
Реализация интерфейса поиска и фильтрации	Frontend-разработчик	30
Тестирование backend-логики	QA-инженер	20
Тестирование интерфейса и функционала	QA-инженер	25
Всего		352

Таким образом, с учетом анализа конкурентов, разработки уникального дизайна, реализации функционала для четырех ролей пользователей, тестирования и развертывания на сервере, затраты рабочего времени составили 352 часов.

6.3.2 Расчет основной заработной платы

Для расчета заработной платы используется два параметра расчета: трудозатраты и заработная плата специалиста в зависимости от вида и рода работ, которые он выполняет. Основная заработная плата отдельного специалиста будет рассчитываться по формуле 6.1.

$$C_{\text{оз}} = (T_{\text{раз}} \cdot C_{\text{зп}}), \quad (6.1)$$

где $T_{\text{раз}}$ – трудоемкость (чел./час.);
 $C_{\text{зп}}$ – средняя часовая ставка руб./час.

На основании анализа рыночных данных о заработных платах в IT-сфере Беларуси, были определены средние часовые ставки для каждого типа специалистов. Расчет основной заработной платы выполнен путем умножения трудозатрат в часах на соответствующую часовую ставку. Общие данные и расчет заработной платы для каждого специалиста представлены в таблице 6.4.

Таблица 6.4 – Общие данные и расчет ЗП специалистов

Исполнитель	Затраты рабочего времени, часов	Средняя часовая ставка, руб./час	Основная заработная плата, руб.
Бизнес-аналитик	47	40,20	1 889,40
Системный архитектор	20	63,34	1 266,80
UX/UI-дизайнер	55	36,60	2 013,00
Frontend-разработчик	70	41,54	2 907,80
Backend-разработчик	80	48,43	3 874,40
DevOps-инженер	35	42,89	1 501,15
QA-инженер	45	32,91	1 480,95
Всего	352		14 933,50

Таким образом, основная заработная плата рассчитана как сумма заработной платы каждого специалиста:

$$C_{\text{оз}} = 1\,889,40 + 1\,266,80 + 2\,013,00 + 2\,907,80 + 3\,874,40 + 1\,501,15 + 1\,480,95 \\ = 14\,933,50 \text{ руб.}$$

Основная заработная плата специалистов составила 14 933,50 рублей.

6.3.3 Расчет дополнительной заработной платы

Дополнительная заработная плата определяется по нормативу в процентах к основной заработной плате по формуле 6.2.

$$C_{\text{дз}} = (C_{\text{оз}} \cdot H_{\text{дз}}) / 100, \quad (6.2)$$

где $H_{\text{дз}}$ – норматив дополнительной заработной платы, равный 8.

Сумма дополнительной заработной платы составляет:

$$C_{дз} = (14\,933,50 \cdot 8) / 100 = 1\,194,68 \text{ руб.}$$

Таким образом, сумма дополнительной заработной платы специалистов составила 1 194,68 рублей.

6.3.4 Расчет отчислений в Фонд социальной защиты населения и по обязательному страхованию

Отчисления в Фонд социальной защиты населения (ФСЗН) и по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний в БРУСП «Белгосстрах» определяются в соответствии с действующими законодательными актами по нормативу в процентном отношении к фонду основной и дополнительной зарплаты исполнителей.

Отчисления в ФСЗН вычисляются по формуле 6.3.

$$C_{фсзн} = ((C_{оз} + C_{дз}) \cdot N_{фсзн}) / 100, \quad (6.3)$$

где $N_{фсзн}$ – норматив отчислений в Фонд социальной защиты населения, %.

$$C_{сзн} = ((14\,933,50 + 1\,194,68) \cdot 34) / 100 = 5\,483,58 \text{ руб.}$$

Отчисления в БРУСП «Белгосстрах» вычисляются по формуле 6.4.

$$C_{бгс} = ((C_{оз} + C_{дз}) \cdot N_{бгс}) / 100, \quad (6.4)$$

где $N_{бгс}$ – ставка отчислений по страхованию в БРУСП «Белгосстрах», %.

Отчисления в Фонд социальной защиты населения (ФСЗН) и по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний в БРУСП «Белгосстрах» составит:

$$C_{бгс} = ((14\,933,50 + 1\,194,68) \cdot 0,6) / 100 = 96,77 \text{ руб.}$$

Таким образом, после проведения расчетов отчисления в Фонд социальной защиты населения составили 5 483,58 рублей, а в БРУСП «Белгосстрах» составили 96,77 рублей.

6.3.5 Расчет суммы прочих прямых затрат

Сумма прочих затрат $C_{пз}$ определяется как произведение основной заработной платы исполнителей на конкретное программное средство $C_{оз}$ на норматив прочих затрат в целом по организации $N_{пз}$.

Сумма прочих затрат вычисляются по формуле 6.5.

$$C_{пз} = (C_{оз} \cdot H_{пз}) / 100, \quad (6.5)$$

где $H_{пз}$ – норматив прочих прямых затрат, %.

Сумма прочих затрат составит:

$$C_{пз} = (14\,933,50 \cdot 21) / 100 = 3\,136,04 \text{ руб.}$$

Норматив прочих затрат в целом по организации $H_{пз}$ относится к исходным данным и составляет 21%. Основная заработная плата была рассчитана ранее. Следовательно, $C_{пз}$ принимает значение 3 136,04 рубля.

6.3.6 Расчет суммы накладных расходов

Сумма накладных расходов $C_{обп,обх}$ – произведение основной заработной платы исполнителей на конкретное программное средство $C_{оз}$ на норматив накладных расходов в целом по организации $H_{нр}$.

Сумма накладных расходов вычисляются по формуле 6.6.

$$C_{обп,обх} = (C_{оз} \cdot H_{нр}) / 100, \quad (6.6)$$

где $H_{нр}$ – норматив накладных расходов, %.

Сумма накладных расходов составит:

$$C_{обп,обх} = (14\,933,50 \cdot 41) / 100 = 6\,122,74 \text{ руб.}$$

Норматив накладных расходов в целом по организации $H_{обп,обх}$ относится к исходным данным и составляет 41%. Основная заработная плата была рассчитана ранее. Следовательно, $C_{обп,обх}$ принимает значение 6 122,74 рублей.

6.3.7 Сумма расходов на разработку программного средства

Сумма расходов на разработку программного средства C_p определяется как сумма основной и дополнительной заработных плат исполнителей на конкретное программное средство, отчислений на социальные нужды, суммы прочих затрат и суммы накладных расходов, по формуле 6.7.

$$C_p = C_{оз} + C_{дз} + C_{фсзн} + C_{бгс} + C_{пз} + C_{обп,обх}, \quad (6.7)$$

Для расчета потребуются значения расходов на разработку, полученных ранее. Подставив значения в формулу (6.7), получаем:

$$C_p = 14\,933,50 + 1\,194,68 + 5\,483,58 + 96,77 + 3\,136,04 + 6\,122,74 = 30\,967,31 \text{ руб.}$$

Таким образом сумма расходов на разработку программного средства была вычислена на основе данных, которые рассчитаны ранее в данном разделе, и составила 30 967,31 рублей.

6.3.8 Расходы на сопровождение и адаптацию

Сумма расходов на сопровождение и адаптацию программного средства $C_{рса}$ как произведение суммы расходов на разработку на норматив расходов на сопровождение и адаптацию $H_{рр}$, и находится по формуле 6.8.

$$C_{рса} = (C_p \cdot H_{рр}) / 100, \quad (6.8)$$

где $H_{рр}$ – норматив расходов на сопровождение и адаптацию, %.

Подставив следующие данные: $C_p = 30\,967,31$ руб., $H_{рр} = 10$ (таблица 6.1) в формулу (6.8), получаем:

$$C_{рса} = 30\,967,31 \cdot 10 / 100 = 3\,096,73 \text{ руб.}$$

Исходя из вышеуказанных расчетов общая сумма расходов на сопровождение и адаптацию составила 3 096,73 рублей. Все проведенные выше расчеты необходимы для вычисления полной себестоимости проекта.

6.3.9 Расчет полной себестоимости

Полная себестоимость $C_{п}$ определяется как сумма двух элементов: суммы расходов на разработку C_p и суммы расходов на сопровождение и адаптацию программного средства $C_{рса}$. Полная себестоимость $C_{п}$ вычисляется по формуле 6.9.

$$C_{п} = C_p + C_{рса}, \quad (6.9)$$

Подставив в формулу (6.9) ранее рассчитанные значения $C_p = 30\,967,31$ руб. и $C_{рса} = 3\,096,73$ руб., получим:

$$C_{п} = 30\,967,31 + 3\,096,73 = 34\,064,04 \text{ руб.}$$

Полная себестоимость программного средства была вычислена на основе данных, рассчитанных ранее в данном разделе, и составила 34 064,04 рублей.

6.3.10 Определение цены, оценка эффективности

Для определения суммы денежных поступлений и окупаемости затрат на разработку веб-приложения для аренды жилья произведен расчет на основе данных о продуктах-аналогах с учетом монетизации через нативную рекламу.

Продукты-аналоги:

– NedvRf это платформа для поиска недвижимости, включая аренду жилья, с базовыми фильтрами по цене и местоположению.

– Tuktuk это сервис краткосрочной аренды жилья в Москве, сосредоточенный на прямом взаимодействии между арендаторами и собственниками.

– Kwartirka это платформа для краткосрочной аренды жилья в российских городах. Предлагает простой поиск по городам и ценам.

Показатели качества:

- функциональность;
- удобство интерфейса;
- локальный фокус;
- безопасность данных.

Расчет показателей качества базового и нового продуктов, согласно балловому методу, приводится в таблице 6.5.

Таблица 6.5 – Оценка качества программного средства

Показатель качества	Весовой коэффициент	Моё приложение	NedvRf	TukTuk	Kvartirka
Функциональность	0,4	6	5	5	6
Удобство интерфейса	0,3	7	6	6	6
Локальный фокус	0,2	9	8	9	9
Безопасность данных	0,1	6	5	5	6
Всего	1	6,9	5,9	6,1	6,6

Расчет прогнозного количества установок программного средства K_1 при монетизации подписочным методом, рассчитывается по формуле 6.10

$$K_1 = (K_0 \cdot \text{ИР}) / (T_0 \cdot \text{ИК}), \quad (6.10)$$

где K_0 – количество установок ПС конкурента;

T_0 – количество лет существования приложения;

ИР – показатель рассматриваемого программного продукта;

ИК – показатель программного продукта конкурента.

$$K1 = (1\,000\,000 / 5 \cdot 6,9) / 5,9 = 233898 \text{ (установок в год),}$$

$$K2 = (800\,000 / 4 \cdot 6,9) / 6,1 = 226230 \text{ (установок в год),}$$

$$K3 = (1\,200\,000 / 6 \cdot 6,9) / 6,6 = 209091 \text{ (установок в год),}$$

$$K = (233898 + 226230 + 209091) / 3 = 223073 \text{ (установок в год).}$$

По данным исследований CTR за тысячу показов составляет в среднем 1%. Ожидается, что в среднем каждый пользователь будет добавлять около 40 ссылок. Реклама будет отображаться как нативная реклама, каждый переход будет оцениваться в 0,1 белорусских рублей. Если среднее количество

установок 223 073 за год, то денежные поступления от рекламы в приложении $P_{\text{ост.в год}} = 8\,922,92$ рублей за год.

Срок окупаемости приложения $T_{\text{ок}}$ вычисляется по формуле 6.11

$$T_{\text{ок}} = C_{\text{п}} / P_{\text{ост.в год}}, \quad (6.11)$$

где $C_{\text{п}}$ – полная себестоимость, руб.;

$P_{\text{ост.в год}}$ – денежные поступления от показа рекламы в приложении за год, руб.

$$T_{\text{ок}} = 34\,064,04 / 8\,922,92 = 3,82 \text{ года.}$$

Таким образом срок окупаемости веб-приложения составляет 3,82 года.

6.4 Выводы по разделу

В таблице 6.6 и в приложении Ж представлены результаты расчетов для основных показателей данной главы в краткой форме.

Таблица 6.6 – Результаты расчетов

Наименование показателя	Значение
Время разработки, ч	352
Основная заработная плата, руб.	14 933,50
Дополнительная заработная плата, руб.	1 194,68
Отчисления в Фонд социальной защиты населения и БРУСП «Белгосстрах», руб.	5 483,58 96,77
Прочие прямые затраты, руб.	3 136,04
Накладные расходы, руб.	6 122,74
Себестоимость разработки программного средства, руб.	30 967,31
Расходы на сопровождение и адаптацию, руб.	3 096,73
Полная себестоимость, руб.	34 064,04
Срок окупаемости, год	3,82

Разработка программного средства, осуществляемая одним программистом в течении 1,5 месяца, при заданных условиях обойдется в 34 064,04. Реализации данного программного средства будет приносить годовые денежные поступления от показа рекламы в размере 8 922,92 белорусских рублей и окупиться через 3,82 года.

Необходимость разработки веб-приложения для аренды жилья обусловлена ростом спроса на удобные и доступные платформы для поиска и бронирования недвижимости, так как все больше людей ищут жилье для краткосрочной и долгосрочной аренды. К приложению предъявлены требования по простоте использования и полноте функционала. Разработанное программное средство решает задачи по организации поиска, бронирования и коммуникации между арендаторами и владельцами жилья.

Заключение

В процессе разработки веб-приложения для аренды жилья была успешно реализована цель создания эффективного инструмента для взаимодействия пользователя и владельца. Для разработки был выбран стек технологий, включающий Node.js, React и PostgreSQL, что обеспечило надежную серверную часть, гибкий и интуитивно понятный интерфейс, а также устойчивое хранение данных.

В ходе выполнения проекта были реализованы ключевые функции:

- регистрация и авторизация пользователей с учетом разных ролей (пользователь, владелец, администратор);
- создание и управление жильем;
- бронирование жилья;
- оставление отзывов после подтверждения бронирования;
- управление категориями и критериями жилья для администратора.

Процесс разработки включал создание базы данных, серверной и клиентской частей приложения, а также интеграцию необходимых функциональных возможностей для обеспечения работы системы. Все функции были тщательно протестированы, и проект прошел все необходимые тесты, подтвердив свою работоспособность и выполнение всех технических требований.

Тестирование показало, что приложение функционирует стабильно, все сценарии работы с жильем и профилями пользователей проходят без ошибок. В ходе тестирования не было выявлено проблем, и все поставленные задачи были успешно реализованы.

Разработанное приложение готово к эксплуатации, и его можно использовать для упрощения аренды жилья, обеспечивая удобный и эффективный интерфейс для всех участников процесса.

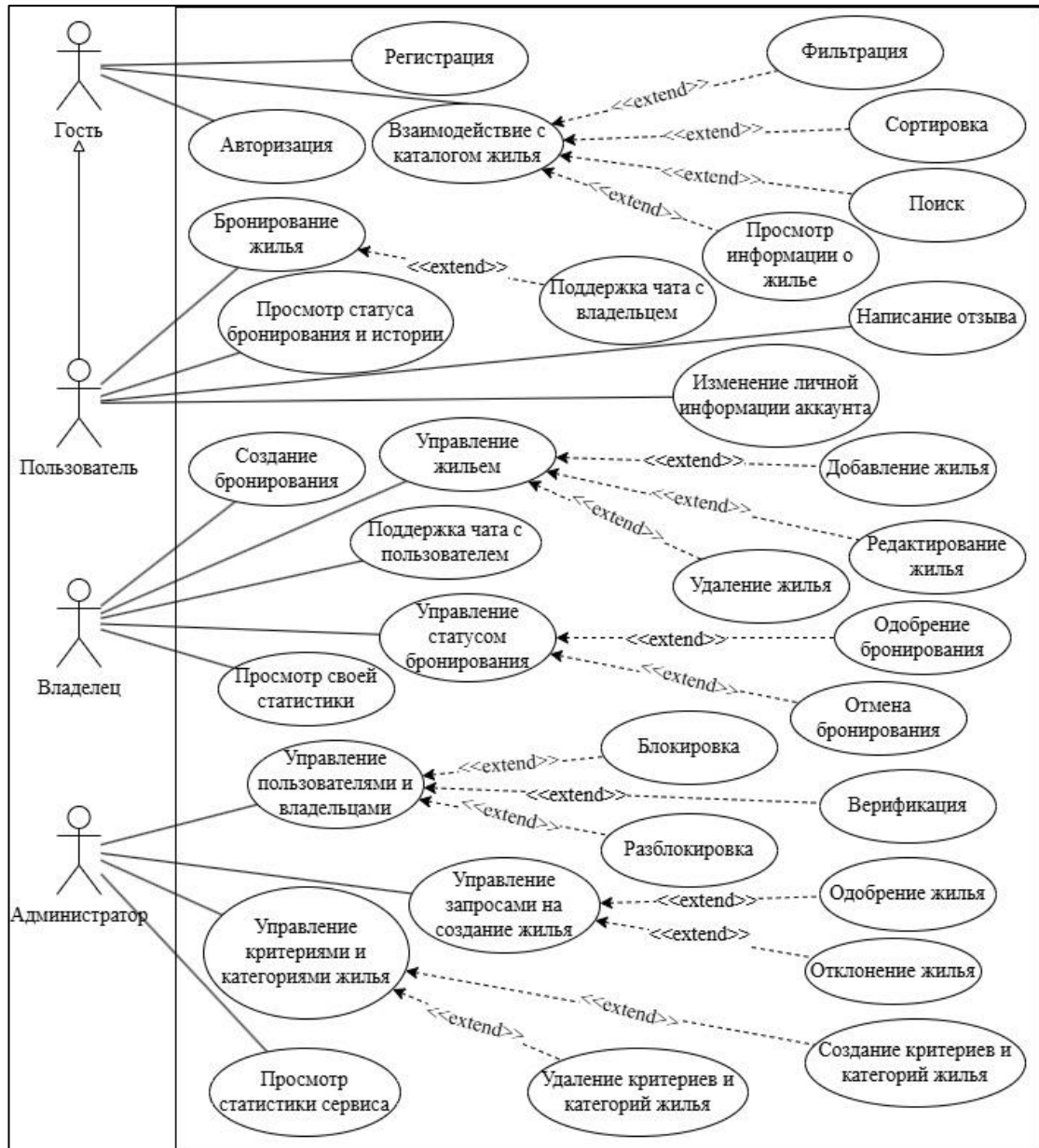
					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Заключение		
Разраб.		Синяк П.С.					
Пров.		Комкова А.В.					
Н. контр.		Алешаускас В.А.					
Утв.		Блинова Е.А.			БГТУ 1-40 05 01, 2025		
		Лит.	Лист	Листов			
		У	1	1			

Список использованных источников

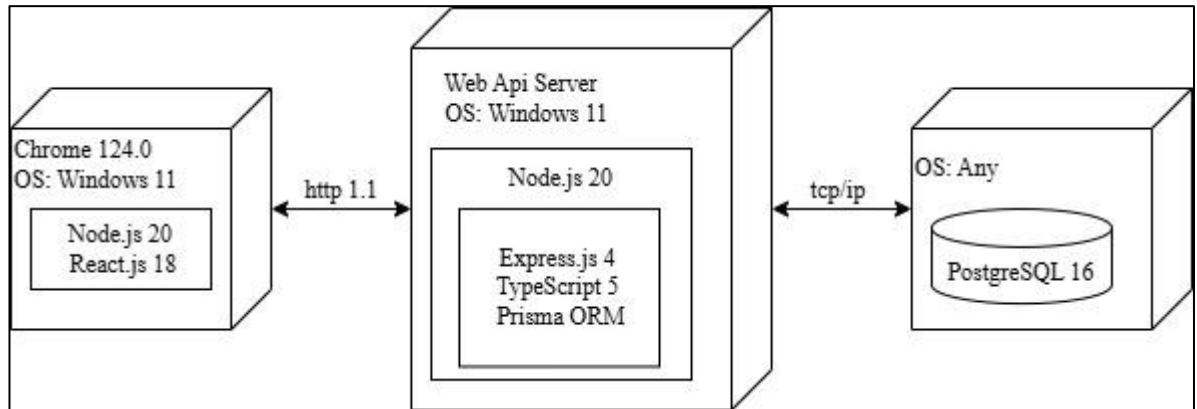
1. Airbnb: [сайт]. – [Dublin], 2008–2025. – URL: <https://www.airbnb.com> (дата обращения: 24.03.2025).
2. ЦИАН: [сайт]. – Москва, 2005–2025. – URL: <https://www.cian.ru> (дата обращения: 24.03.2025).
3. Суточно.ру: [сайт]. – Москва, 2011–2025. – URL: <https://sutochno.ru> (дата обращения: 24.03.2025).
4. Visual Studio Code: [сайт]. – [Редмонд], 2015–2025. – URL: <https://code.visualstudio.com> (дата обращения: 24.03.2025).
5. Node.js: [сайт]. – [Сан-Франциско], 2009–2025. – URL: <https://nodejs.org> (дата обращения: 25.03.2025).
6. TypeScript: [сайт]. – [Редмонд], 2012–2025. – URL: <https://www.typescriptlang.org> (дата обращения: 26.03.2025).
7. METANIT.COM: учебный сайт по программированию: [сайт]. – [Москва], 2014–2025. – URL: <https://metanit.com/web/react/> (дата обращения: 27.03.2025).
8. Postgres: документация PostgreSQL: [сайт]. – Москва, 2015–2025. – URL: <https://postgrespro.ru/docs/postgresql.com> (дата обращения: 28.03.2025).

					ДП 00.00.ПЗ		
		ФИО	Подпись	Дата	Список использованных источников		
Разраб.	Синяк П.С.						
Пров.	Комкова А.В.						
Н. контр.	Алешаускас В.А.						
Утв.	Блинова Е.А.				БГТУ 1-40 05 01, 2025		
					Лит.	Лист	Листов
					У	1	1

ПРИЛОЖЕНИЕ А Диаграмма вариантов использования

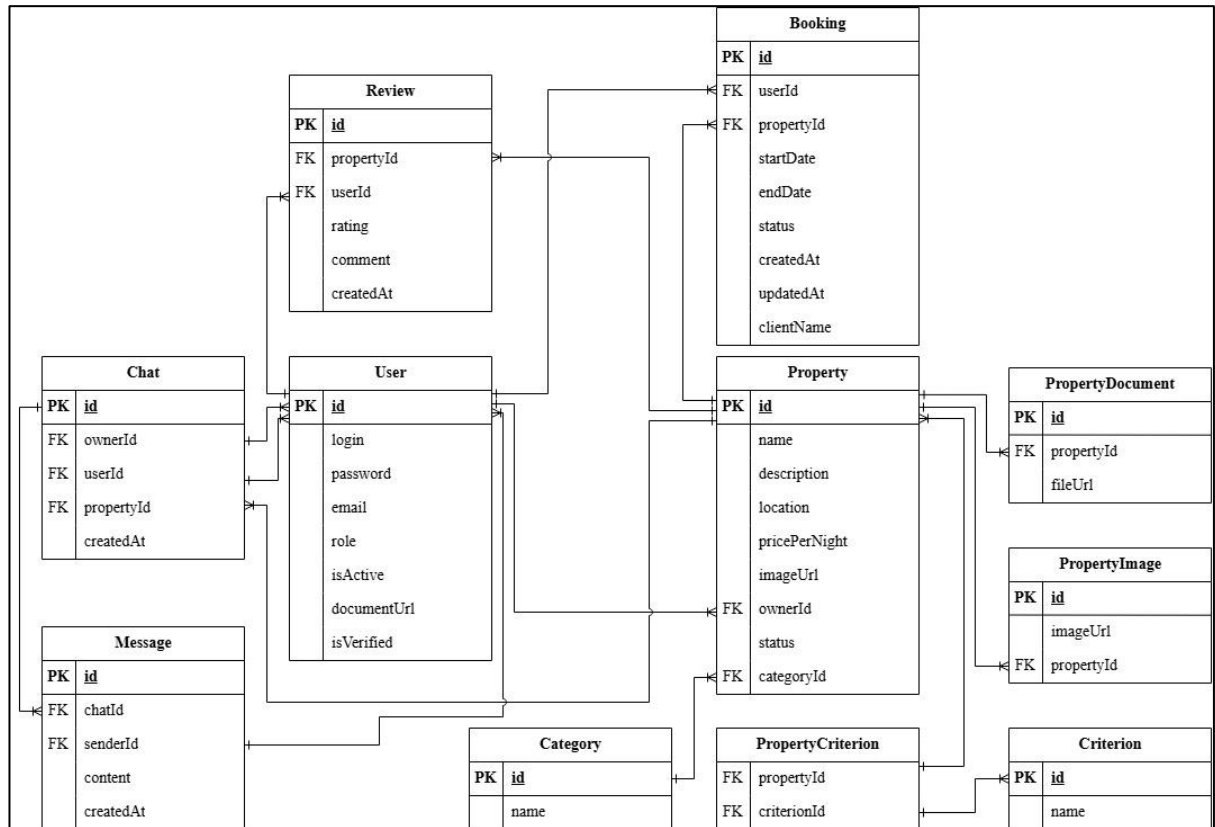


ПРИЛОЖЕНИЕ Б
Архитектурная схема веб-приложения

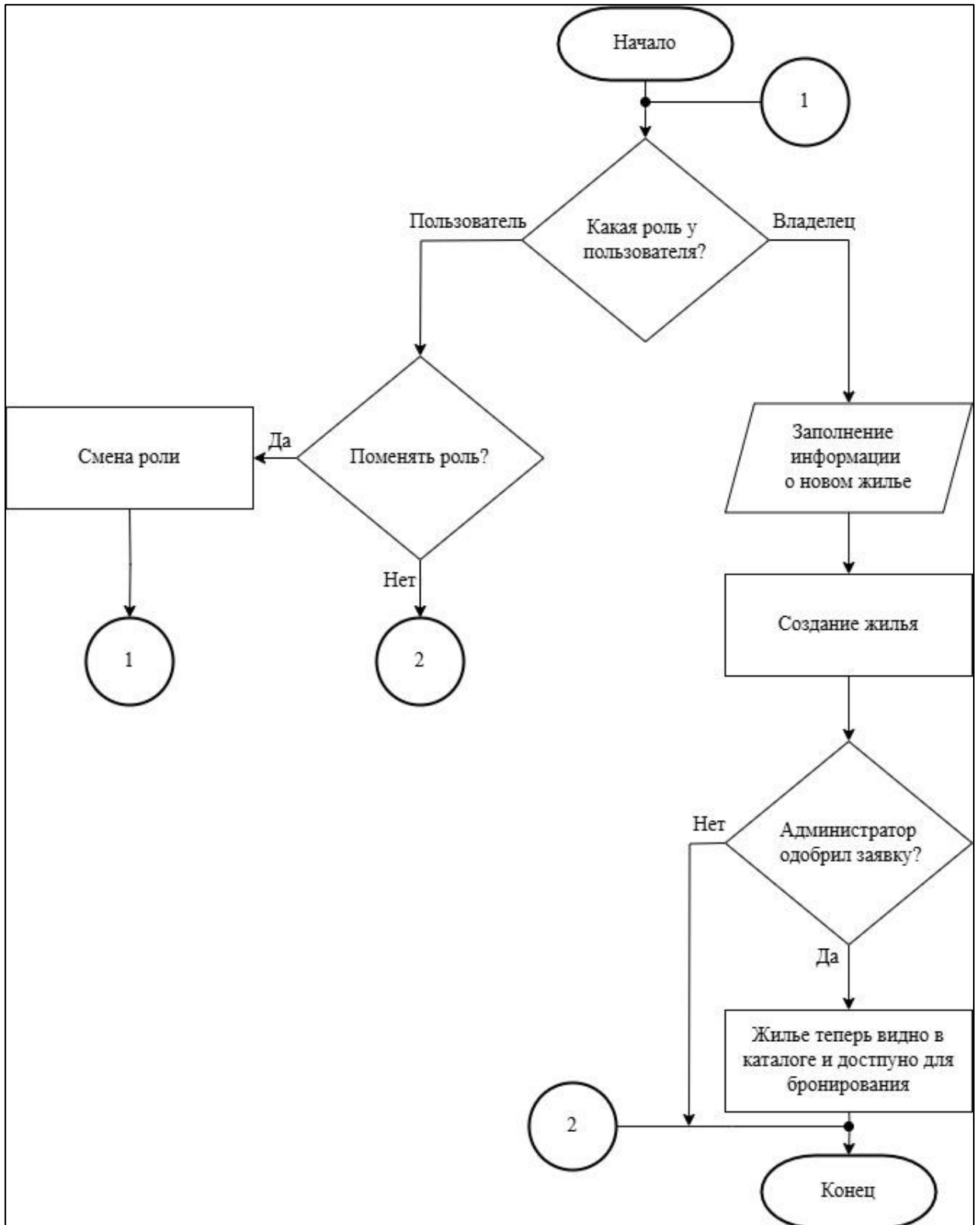


ПРИЛОЖЕНИЕ В

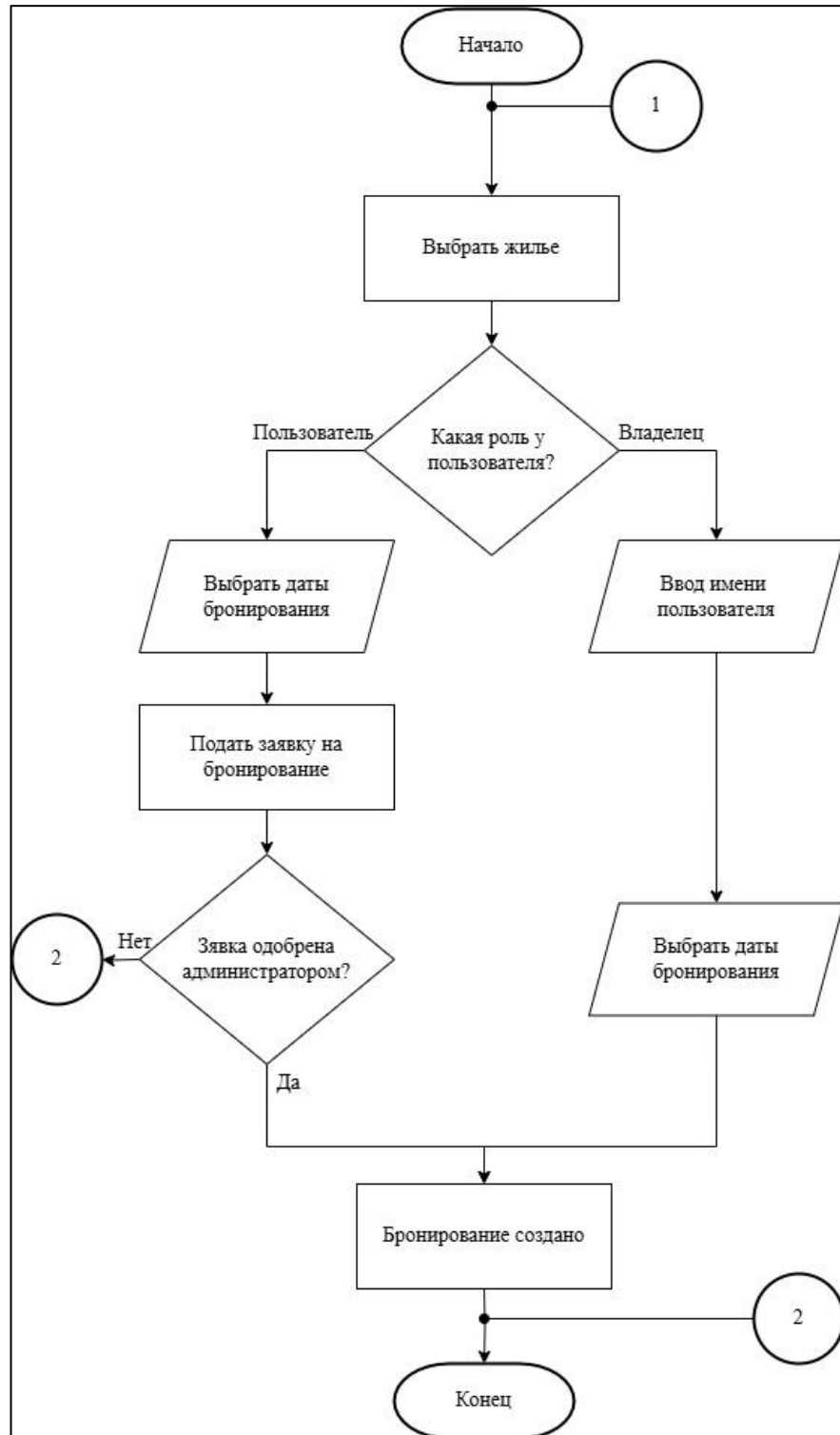
Логическая схема базы данных



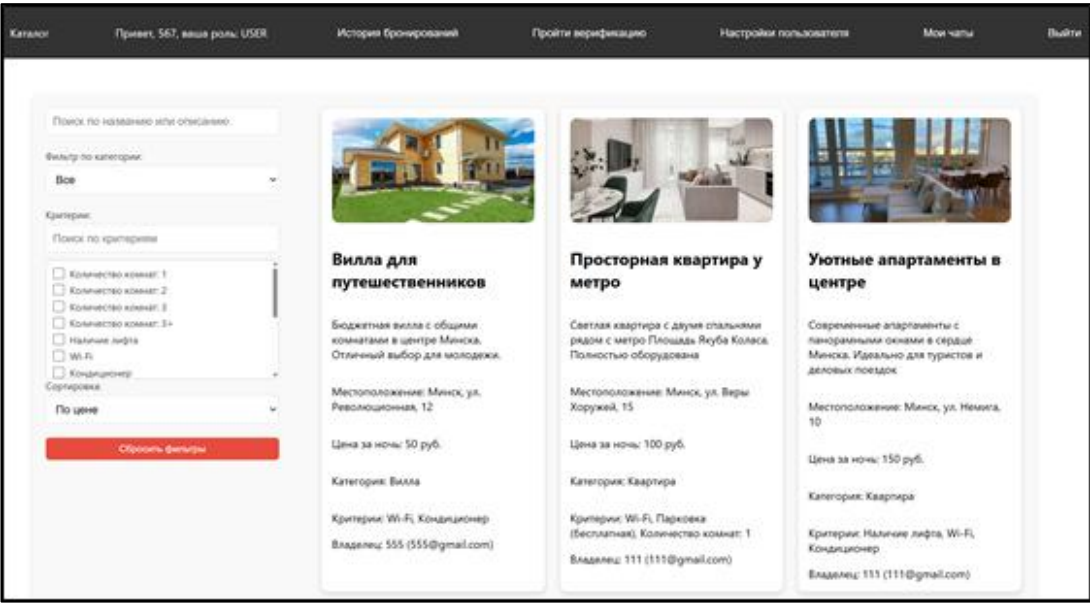
ПРИЛОЖЕНИЕ Г
Блок-схема алгоритма добавления жилья



ПРИЛОЖЕНИЕ Д
Блок-схема алгоритма бронирования жилья



ПРИЛОЖЕНИЕ Е
Скриншот рабочего веб-приложения



ПРИЛОЖЕНИЕ Ж
Таблица экономических показателей

Наименование показателя	Значение
Время разработки, ч	352
Основная заработная плата, руб.	14933,50
Дополнительная заработная плата, руб.	1194,68
Отчисления в Фонд социальной защиты населения и БРУСП «Белгосстрах», %, руб.	5483,58 96,77
Прочие прямые затраты, руб.	3136,04
Накладные расходы, руб.	6122,74
Себестоимость разработки программного средства, руб.	30967,31
Расходы на сопровождение и адаптацию, руб.	3096,73
Полная себестоимость, руб.	34064,04
Срок окупаемости, год	3,82